

COIMBATORE INSTITUTE OF TECHNOLOGY
(An Autonomous Institution Affiliated to Anna University)

COIMBATORE – 14

Network Programming Lab



ROLL NO	: 2303717672621002
NAME	: AANIRUDH.P
CLASS	: MSC (Software Systems)
SUBJECT CODE	: 20MSS56
BATCH	: 2023 - 2028

INDEX

Ex No	Date	Index	Marks
1	18.06.2025	TCP : Iterative Client-Server Implementation	
2	25.06.2025	TCP Iterative Server	
3	02.07.2025	UDP : Echo Client - Server Implementation	
4	08.07.2025	UDP : Time Server Implementation	
5	09.07.2025	TCP TIME Server	
6	22.07.2025	TCP Concurrent Server	
7	02.08.2025	TCP Concurrent Server Application	
8	20.08.2025	Broadcasting	
9	20.08.2025	Port Scanner	
10	01.09.2025	Simulation of DHCP Server	
11	08.09.2025	Multicasting	
12	09.09.2025	Multicasting:Quiz Application	
13	15.09.2025	Raw Sockets:IP Header	
14	22.09.2025	Raw Sockets:TCP Header	
15	29.09.2025	Packet Sniffer	
16	06.10.2025	I/O Multiplexing	

EX.NO: 1

TCP – Client Server Implementation

DATE: 01-06-2025

AIM:

Develop an Iterative TCP Client -

- Server application. Server accepts a connection, reads data and echos it to the client.
- Client opens a connection with the server, sends a message and reads the echo and prints it on the screen.
- Print the time required for establishing the connection and the time to receive the echo.
- Test the application across machines.

DESCRIPTION:

Server Description

The server program creates a TCP socket, binds it to a specific port, and listens for incoming client connections.

When a client connects, it receives a message from the client, prints it on the server console, and sends the same message back (echo).

It handles one client at a time in an iterative loop and waits for the next client after finishing with the current one.

Client Description

The client program creates a TCP socket and connects to the server using the server's IP address and port.

It records the time taken to establish the connection, sends a predefined message to the server, and waits for the echoed message.

It records the time taken to receive the echo, then displays the echoed message along with the measured connection time and echo time.

CODE:

Server.c

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#define PORT 8080

#define BUFFER_SIZE 1024

int main() {

    int server_fd, client_fd;

    struct sockaddr_in server_addr, client_addr;

    char buffer[BUFFER_SIZE];

    socklen_t addr_len = sizeof(client_addr);

    // Create socket

    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {

        perror("Socket failed");

        exit(EXIT_FAILURE);

    }

    // Bind

    server_addr.sin_family = AF_INET;

    server_addr.sin_addr.s_addr = INADDR_ANY; // Accept connections from any interface

    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {

        perror("Bind failed");

        close(server_fd);

        exit(EXIT_FAILURE);

    }
```

```
// Liste
if (listen(server_fd, 5) < 0) {
    perror("Listen failed");
    close(server_fd);
    exit(EXIT_FAILURE);
}

printf("Server listening on port %d...\n", PORT);

while (1) {
    // Accept client connection
    client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);
    if (client_fd < 0) {
        perror("Accept failed");
        continue;
    }

    printf("Connected to client: %s\n", inet_ntoa(client_addr.sin_addr));

    // Read data from client
    int bytes_read = read(client_fd, buffer, BUFFER_SIZE);
    if (bytes_read > 0) {
        buffer[bytes_read] = '\0';
        printf("Received: %s\n", buffer);

        // Echo back to client
        send(client_fd, buffer, strlen(buffer), 0);
    }
    close(client_fd);
}
```

```
    close(server_fd);  
    return 0;  
}
```

Client.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <arpa/inet.h>  
#include <time.h>
```

```
#define PORT 8080  
#define BUFFER_SIZE 1024
```

```
int main() {  
    int sock;  
    struct sockaddr_in server_addr;  
    char buffer[BUFFER_SIZE] = "Hello from client!";  
    char recv_buffer[BUFFER_SIZE];  
    struct timespec start_conn, end_conn, start_echo, end_echo;
```

```
    // Create socket
```

```
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {  
        perror("Socket failed");  
        exit(EXIT_FAILURE);  
    }
```

```
    server_addr.sin_family = AF_INET;
```

```
server_addr.sin_port = htons(PORT);

// Change this IP to your server's IP
if (inet_pton(AF_INET, "10.251.236.191", &server_addr.sin_addr) <= 0) {
    perror("Invalid address");
    exit(EXIT_FAILURE);
}

// Measure connection time
clock_gettime(CLOCK_MONOTONIC, &start_conn);
if (connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
    perror("Connection failed");
    close(sock);
    exit(EXIT_FAILURE);
}
clock_gettime(CLOCK_MONOTONIC, &end_conn);

// Send message
send(sock, buffer, strlen(buffer), 0);

// Measure echo receive time
clock_gettime(CLOCK_MONOTONIC, &start_echo);
int bytes_received = read(sock, recv_buffer, BUFFER_SIZE);
clock_gettime(CLOCK_MONOTONIC, &end_echo);

if (bytes_received > 0) {
    recv_buffer[bytes_received] = '\0';
    printf("Echo from server: %s\n", recv_buffer);
}

// Calculate times
```

```
double conn_time = (end_conn.tv_sec - start_conn.tv_sec) +  
    (end_conn.tv_nsec - start_conn.tv_nsec) / 1e9;  
double echo_time = (end_echo.tv_sec - start_echo.tv_sec) +  
    (end_echo.tv_nsec - start_echo.tv_nsec) / 1e9;  
  
printf("Connection time: %.6f seconds\n", conn_time);  
printf("Echo receive time: %.6f seconds\n", echo_time);  
  
close(sock);  
return 0;  
}
```


OUTPUT:

```
aanirudh@aanirudh-VirtualBox: ~/Desktop
inet 127.0.0.1/8 scope host lo
    valid lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
    valid lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    link/ether 08:00:27:a0:b9:37 brd ff:ff:ff:ff:ff:ff
    inet 10.251.236.191/24 brd 10.251.236.255 scope global dynamic noprefixroute
    enp0s3
        valid lft 3564sec preferred_lft 3564sec
    inet6 fe80::296c:1409:57b1:40fe/64 scope link noprefixroute
        valid lft forever preferred_lft forever
aanirudh@aanirudh-VirtualBox:~$ cd Desktop
aanirudh@aanirudh-VirtualBox:~/Desktop$ gcc server.c -o se
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/13/../../../../x86_64-linux-gnu/Scrt1.o:
in function `start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
aanirudh@aanirudh-VirtualBox:~/Desktop$ gcc server.c -o se
aanirudh@aanirudh-VirtualBox:~/Desktop$ ./se
Server listening on port 8080...
Connected to client: 10.251.236.191
Received: Hello from client!
```

```
aanirudh@aanirudh-VirtualBox:~/Desktop
aanirudh@aanirudh-VirtualBox:~$ cd Desktop
aanirudh@aanirudh-VirtualBox:~/Desktop$ gcc client.c -o ck
aanirudh@aanirudh-VirtualBox:~/Desktop$ ./ck
Echo from server: Hello from client!
Connection time: 0.000142 seconds
Echo receive time: 0.001342 seconds
aanirudh@aanirudh-VirtualBox:~/Desktop$
```

EX.NO: 2

TCP- Iterative Hangman Server

DATE: 02-07-2025

AIM:

Develop TCP Based Hangman Server. Develop a Client Application which allows the user to play with the Client.

The user will lose the game if either the allotted time runs out or all available chances are used up .

DESCRIPTION:

TCP Hangman Server – Description

A TCP-based program that waits for a client to connect, then plays a Hangman game.

- Creates a socket, binds to a port, and listens for connections.
- Picks a random word, hides it with underscores, and tracks guesses, chances, and time.
- Receives letters from the client, updates the game state, and sends it back.
- Ends the game when the word is guessed, chances are over, or time runs out.

TCP Hangman Client – Description

A TCP program that connects to the Hangman server and plays the game.

- Connects to the server via IP and port.
- Receives the masked word, chances, and time.
- Sends guessed letters to the server and displays the updated game state.
- Shows win/loss message and closes the connection at the end.

CODE:

Hangman_server.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <time.h>


#define PORT 8080

#define BUFFER_SIZE 1024

#define MAX_CHANCES 6

#define TIME_LIMIT 30 // seconds


// Word list
const char *words[] = {"network", "socket", "virtual", "hangman", "protocol"};


void mask_word(const char *word, char *masked) {
    for (int i = 0; word[i] != '\0'; i++) {
        masked[i] = '_';
    }
    masked[strlen(word)] = '\0';
}


int update_mask(const char *word, char *masked, char guess) {
    int correct = 0;
    for (int i = 0; word[i] != '\0'; i++) {
        if (word[i] == guess && masked[i] == '_') {
            masked[i] = guess;
            correct = 1;
        }
    }
    return correct;
}
```

```
int main() {  
    int server_fd, client_fd;  
    struct sockaddr_in server_addr, client_addr;  
    socklen_t addr_len = sizeof(client_addr);  
    char buffer[BUFFER_SIZE];  
    srand(time(NULL));  
  
    // Create socket  
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {  
        perror("Socket failed");  
        exit(EXIT_FAILURE);  
    }  
  
    server_addr.sin_family = AF_INET;  
    server_addr.sin_addr.s_addr = INADDR_ANY;  
    server_addr.sin_port = htons(PORT);  
  
    if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {  
        perror("Bind failed");  
        close(server_fd);  
        exit(EXIT_FAILURE);  
    }  
  
    if (listen(server_fd, 5) < 0) {  
        perror("Listen failed");  
        close(server_fd);  
        exit(EXIT_FAILURE);  
    }  
  
    printf("Hangman Server listening on port %d...\n", PORT);
```

```
while (1) {  
    client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);  
    if (client_fd < 0) {  
        perror("Accept failed");  
        continue;  
    }  
  
    printf("Client connected: %s\n", inet_ntoa(client_addr.sin_addr));  
  
    // Pick random word  
    const char *word = words[rand() % (sizeof(words) / sizeof(words[0]))];  
    char masked[BUFFER_SIZE];  
    mask_word(word, masked);  
  
    int chances = MAX_CHANCES;  
    time_t start_time = time(NULL);  
  
    // Send initial game state  
    snprintf(buffer, sizeof(buffer), "WORD: %s Chances: %d Time: %d", masked, chances,  
TIME_LIMIT);  
    send(client_fd, buffer, strlen(buffer), 0);  
  
    // Game loop  
    while (1) {  
        memset(buffer, 0, sizeof(buffer));  
        int bytes_read = read(client_fd, buffer, sizeof(buffer));  
        if (bytes_read <= 0) break;  
  
        char guess = buffer[0];  
        time_t now = time(NULL);
```

```

// Check time limit
if (difftime(now, start_time) > TIME_LIMIT) {
    snprintf(buffer, sizeof(buffer), "TIMEOUT! You lose. Word was: %s", word);
    send(client_fd, buffer, strlen(buffer), 0);
    break;
}

// Update word
if (!update_mask(word, masked, guess)) {
    chances--;
}

// Check win/loss
if (strcmp(word, masked) == 0) {
    snprintf(buffer, sizeof(buffer), "WIN! The word was: %s", word);
    send(client_fd, buffer, strlen(buffer), 0);
    break;
} else if (chances <= 0) {
    snprintf(buffer, sizeof(buffer), "LOSE! The word was: %s", word);
    send(client_fd, buffer, strlen(buffer), 0);
    break;
} else {
    snprintf(buffer, sizeof(buffer), "WORD: %s Chances: %d Time left: %d",
            masked, chances, TIME_LIMIT - (int)difftime(now, start_time));
    send(client_fd, buffer, strlen(buffer), 0);
}
}

close(client_fd);
printf("Client disconnected.\n");
}

```

```
    close(server_fd);  
    return 0;  
}
```

Hangman_client.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <arpa/inet.h>
```

```
#define PORT 8080
```

```
#define BUFFER_SIZE 1024
```

```
int main() {  
    int sock;  
    struct sockaddr_in server_addr;  
    char buffer[BUFFER_SIZE];  
    char guess;  
  
    // Create socket  
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {  
        perror("Socket failed");  
        exit(EXIT_FAILURE);  
    }
```

```
    server_addr.sin_family = AF_INET;  
    server_addr.sin_port = htons(PORT);
```

```
// Change to server's IP address
if (inet_pton(AF_INET, "10.251.236.191", &server_addr.sin_addr) <= 0) {
    perror("Invalid address/Address not supported");
    exit(EXIT_FAILURE);
}

// Connect to server
if (connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("Connection failed");
    exit(EXIT_FAILURE);
}

// Initial game state
memset(buffer, 0, sizeof(buffer));
read(sock, buffer, sizeof(buffer));
printf("%s\n", buffer);

// Game loop
while (1) {
    printf("Enter a letter: ");
    scanf(" %c", &guess);

    send(sock, &guess, 1, 0);

    memset(buffer, 0, sizeof(buffer));
    int bytes = read(sock, buffer, sizeof(buffer));
    if (bytes <= 0) break;

    printf("%s\n", buffer);

    if (strstr(buffer, "WIN!") || strstr(buffer, "LOSE!") || strstr(buffer, "TIMEOUT!")) {
```

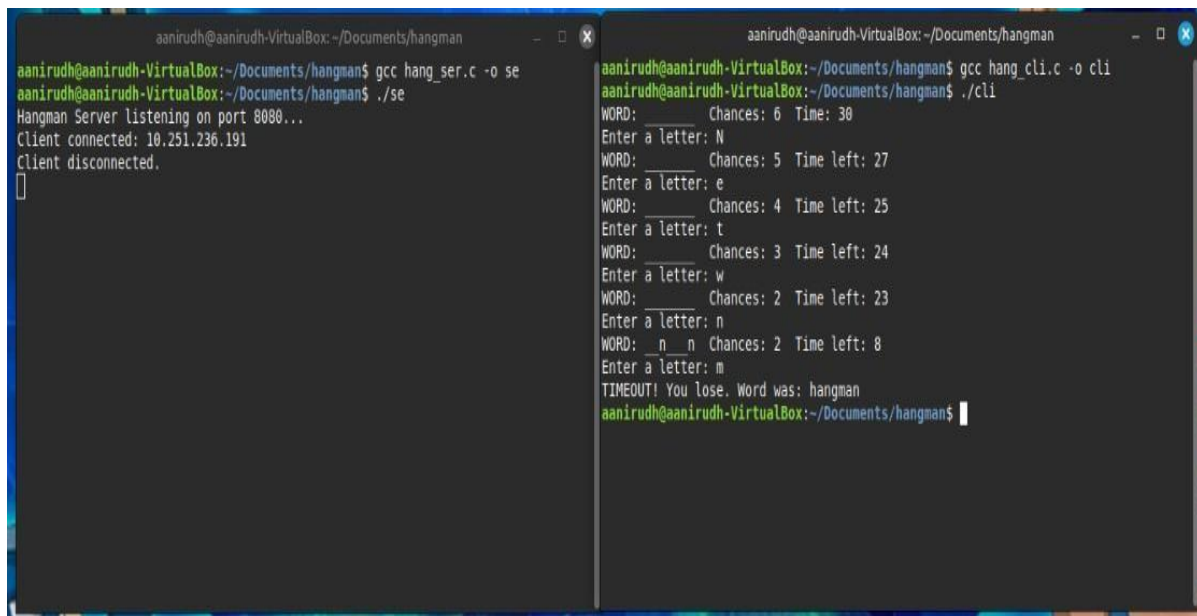


```
        break;
    }
}

close(sock);

return 0;
}
```

OUTPUT:



```
aanirudh@aanirudh-VirtualBox: ~/Documents/hangman
aanirudh@aanirudh-VirtualBox:~/Documents/hangman$ gcc hang_ser.c -o se
aanirudh@aanirudh-VirtualBox:~/Documents/hangman$ ./se
Hangman Server listening on port 8080...
Client connected: 10.251.236.191
Client disconnected.

```

```
aanirudh@aanirudh-VirtualBox: ~/Documents/hangman
aanirudh@aanirudh-VirtualBox:~/Documents/hangman$ gcc hang_cli.c -o cli
aanirudh@aanirudh-VirtualBox:~/Documents/hangman$ ./cli
WORD: _____ Chances: 6 Time: 30
Enter a letter: N
WORD: _____ Chances: 5 Time left: 27
Enter a letter: e
WORD: _____ Chances: 4 Time left: 25
Enter a letter: t
WORD: _____ Chances: 3 Time left: 24
Enter a letter: w
WORD: _____ Chances: 2 Time left: 23
Enter a letter: n
WORD: _n_n Chances: 2 Time left: 8
Enter a letter: m
TIMEOUT! You lose. Word was: hangman
aanirudh@aanirudh-VirtualBox:~/Documents/hangman$
```

EX.NO: 3

UDP – Echo Client Server Implementation

DATE: 02-07-2025

AIM:

UDP : Echo Client - Server Implementation

- Develop UDP echo Client - Server application.
- Test running Client and Server in two different Machines.

DESCRIPTION:

UDP Echo Server

- Creates a UDP socket and binds it to a specific IP and port.
- Waits to receive messages from any client.
- On receiving a message, sends the exact same message back to the sender (echo).
- Keeps running to handle multiple requests (iterative).

UDP Echo Client

- Creates a UDP socket and specifies the server's IP and port.
- Sends a message to the server.
- Waits for the echoed message from the server and displays it.
- Closes the connection after completion.

CODE:

Udp_server.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>

#include <arpa/inet.h>

#define PORT 8080

#define BUFFER_SIZE 1024

int main() {
    int sockfd;

    char buffer[BUFFER_SIZE];

    struct sockaddr_in server_addr, client_addr;

    socklen_t addr_len = sizeof(client_addr);

    // Create UDP socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Server address configuration
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY; // Listen on all interfaces
    server_addr.sin_port = htons(PORT);

    // Bind socket to IP and port
    if (bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
```

```

printf("UDP Echo Server running on port %d...\n", PORT);

while (1) {
    // Receive message from client
    int n = recvfrom(sockfd, buffer, BUFFER_SIZE, 0,
                    (struct sockaddr *)&client_addr, &addr_len);
    buffer[n] = '\0';
    printf("Client: %s\n", buffer);

    // Echo message back
    sendto(sockfd, buffer, strlen(buffer), 0,
           (struct sockaddr *)&client_addr, addr_len);
    printf("Echoed back: %s\n", buffer);
}

close(sockfd);
return 0;
}

```

Udp_client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define SERVER_IP "127.0.0.1" // Change to server's IP for different machines
#define PORT 8080
#define BUFFER_SIZE 1024

```

```

int main() {

    int sockfd;

    char buffer[BUFFER_SIZE];

    struct sockaddr_in server_addr;

    socklen_t addr_len = sizeof(server_addr);


    // Create UDP socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {

        perror("Socket creation failed");

        exit(EXIT_FAILURE);

    }


    // Server address configuration
    memset(&server_addr, 0, sizeof(server_addr));

    server_addr.sin_family = AF_INET;

    server_addr.sin_port = htons(PORT);

    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);


    printf("Enter message: ");

    fgets(buffer, BUFFER_SIZE, stdin);

    buffer[strcspn(buffer, "\n")] = '\0'; // Remove newline


    // Send message to server
    sendto(sockfd, buffer, strlen(buffer), 0,

        (struct sockaddr *)&server_addr, addr_len);


    // Receive echo from server
    int n = recvfrom(sockfd, buffer, BUFFER_SIZE, 0,

        (struct sockaddr *)&server_addr, &addr_len);

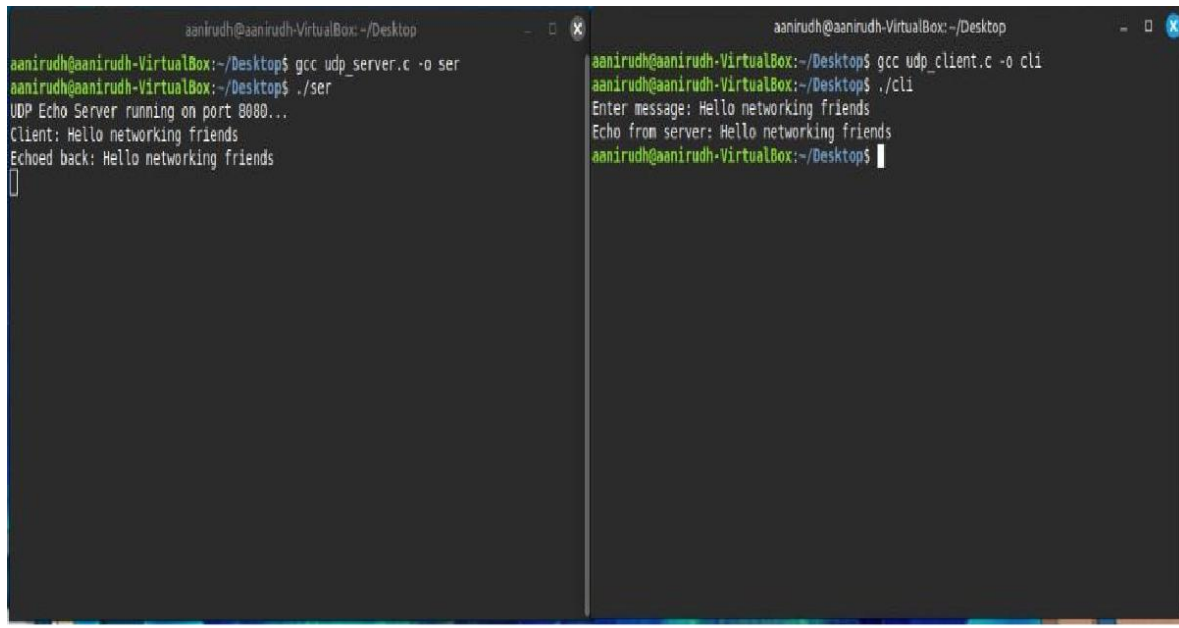
    buffer[n] = '\0';

    printf("Echo from server: %s\n", buffer);
}

```

```
close(sockfd);  
  
return 0;  
}
```

OUTPUT:



The image shows two terminal windows side-by-side, both titled 'aanirudh@aanirudh-VirtualBox: ~/Desktop'. The left window shows the compilation and execution of a UDP server. The right window shows the compilation and execution of a UDP client. The output of the client shows it sending a message and receiving an echo from the server.

```
aanirudh@aanirudh-VirtualBox: ~/Desktop  
aanirudh@aanirudh-VirtualBox:~/Desktop$ gcc udp_server.c -o ser  
aanirudh@aanirudh-VirtualBox:~/Desktop$ ./ser  
UDP Echo Server running on port 8080...  
Client: Hello networking friends  
Echoed back: Hello networking friends  
[  
  
aanirudh@aanirudh-VirtualBox: ~/Desktop  
aanirudh@aanirudh-VirtualBox:~/Desktop$ gcc udp_client.c -o cli  
aanirudh@aanirudh-VirtualBox:~/Desktop$ ./cli  
Enter message: Hello networking friends  
Echo from server: Hello networking friends  
aanirudh@aanirudh-VirtualBox:~/Desktop$
```

EX.NO: 4

UDP Time Server Implementation

DATE: 08-07-2025

AIM:-

UDP : Time Server Implementation

- Develop UDP Time Server application. Check Time.h.
- Test the server with a client.

DESCRIPTION:

UDP Time Server

- The server waits for incoming UDP packets from clients.
- When it receives a request (any message), it gets the current system time using functions from <time.h>.
- It sends the time string back to the requesting client.
- It does not maintain a continuous connection — UDP is connectionless.

UDP Time Client

- The client sends a small request message to the server's IP and port via UDP.
- It then waits for the server's response.
- When it receives the message, it prints the server's current time.
- The server and client can run on different machines as long as they are in the same network.

CODE:

Udp_time_server.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>

#include <time.h>


#define PORT 8080

#define BUFFER_SIZE 1024


int main() {
    int sockfd;

    char buffer[BUFFER_SIZE];

    struct sockaddr_in server_addr, client_addr;

    socklen_t addr_len = sizeof(client_addr);


    // Create UDP socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }


    // Configure server address
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);


    // Bind socket
    if (bind(sockfd, (const struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
```



```

printf("UDP Time Server running on port %d...\n", PORT);

while (1) {
    int n = recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&client_addr, &addr_len);
    if (n < 0) {
        perror("recvfrom failed");
        continue;
    }

    buffer[n] = '\0';

    printf("Request received from %s:%d\n", inet_ntoa(client_addr.sin_addr),
ntohs(client_addr.sin_port));

    // Get current time
    time_t now = time(NULL);
    struct tm *local_time = localtime(&now);
    char time_str[BUFFER_SIZE];
    strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", local_time);

    // Send time to client
    sendto(sockfd, time_str, strlen(time_str), 0, (struct sockaddr *)&client_addr, addr_len);
    printf("Sent time: %s\n", time_str);
}

close(sockfd);
return 0;
}

```

Udp_time_client.c

```
#include <stdio.h>
```

```
#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#define PORT 8080

#define BUFFER_SIZE 1024

int main() {

    int sockfd;

    char buffer[BUFFER_SIZE];

    struct sockaddr_in servaddr;

    // Create UDP socket

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {

        perror("Socket creation failed");

        exit(EXIT_FAILURE);

    }

    // Server details

    servaddr.sin_family = AF_INET;

    servaddr.sin_port = htons(PORT);

    servaddr.sin_addr.s_addr = inet_addr("10.251.236.191"); // Your server's IP

    // Send request to server

    char *message = "TIME_REQUEST";

    sendto(sockfd, message, strlen(message), 0, (struct sockaddr *)&servaddr, sizeof(servaddr));

    printf("Request sent to server...\n");

    // Receive response from server

    socklen_t len = sizeof(servaddr);
```

```
int n = recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&servaddr, &len);

buffer[n] = '\0';

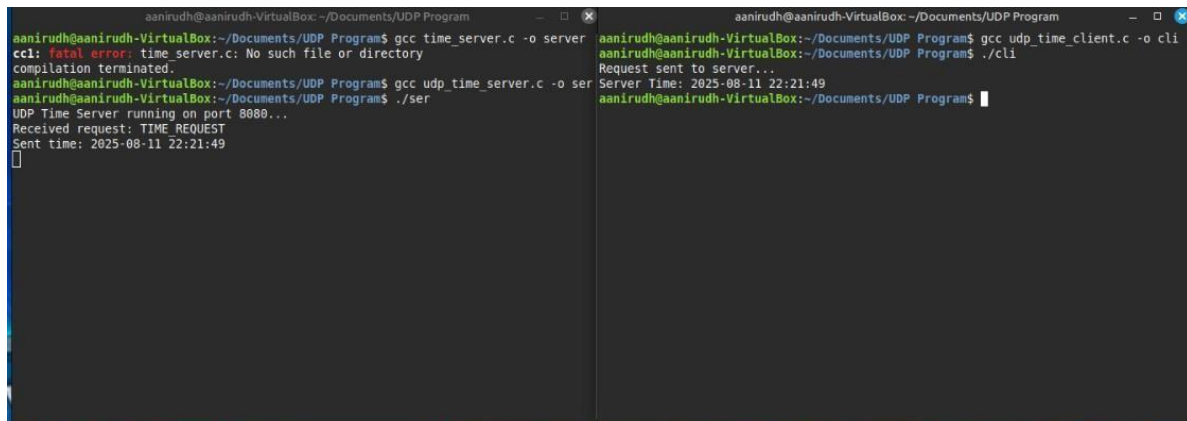
printf("Server Time: %s\n", buffer);

close(sockfd);

return 0;

}
```

OUTPUT:



The image shows two terminal windows side-by-side. The left window shows the compilation of 'time_server.c' with a 'fatal error' message, followed by the successful compilation of 'udp_time_server.c' and its execution. The output shows the server running on port 8080 and receiving a 'TIME REQUEST' at 2025-08-11 22:21:49. The right window shows the compilation of 'udp_time_client.c' and its execution, which sends a request to the server and receives the response 'Server Time: 2025-08-11 22:21:49'.

```
aanirudh@aanirudh-VirtualBox: ~/Documents/UDP Program
aanirudh@aanirudh-VirtualBox:~/Documents/UDP Program$ gcc time_server.c -o server
cc1: fatal error: time_server.c: No such file or directory
compilation terminated.
aanirudh@aanirudh-VirtualBox:~/Documents/UDP Program$ gcc udp_time_server.c -o ser
aanirudh@aanirudh-VirtualBox:~/Documents/UDP Program$ ./ser
UDP Time Server running on port 8080...
Received request: TIME REQUEST
Sent time: 2025-08-11 22:21:49
^C

aanirudh@aanirudh-VirtualBox: ~/Documents/UDP Program
aanirudh@aanirudh-VirtualBox:~/Documents/UDP Program$ gcc udp_time_client.c -o cli
aanirudh@aanirudh-VirtualBox:~/Documents/UDP Program$ ./cli
Request sent to server...
Server Time: 2025-08-11 22:21:49
aanirudh@aanirudh-VirtualBox:~/Documents/UDP Program$
```

EX.NO: 5

TCP Time Server

DATE: 09-07-2025

AIM:

- Develop a DayTime Client and connect to time.nist.gov
- Print the Day and Time

DESCRIPTION:

TCP Time Server — Description

- Creates a TCP socket bound to **10.251.236.191** on port **8000**.
- Waits for a client connection (listen + accept).
- Once connected, gets the **current UTC time** and also calculates **IST time** (UTC + 5 hours 30 minutes).
- Formats both times into a string and sends it to the client.
- Closes the connection after sending.

TCP Time Client — Description

- Resolves the server IP (10.251.236.191) and connects to it via TCP port **8000**.
- Reads the data (UTC and IST time) sent by the server.
- Displays:
 - Server name
 - Server IP address
 - Address type
 - Received time string from the server
- Closes the connection.

CODE:

Time_server.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <time.h>
```

```
#include <netinet/in.h>
```

```
#include <arpa/inet.h>
```

```
#define PORT 8000
```

```
#define BUFFER_SIZE 256
```

```
int main() {
```

```
    int sockfd, newsockfd;
```

```
    struct sockaddr_in server_addr, client_addr;
```

```
    socklen_t client_len = sizeof(client_addr);
```

```
    char buffer[BUFFER_SIZE];
```

```
    time_t rawtime;
```

```
    struct tm *timeinfo;
```

```
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

```
    if (sockfd < 0) {
```

```
        perror("Socket creation failed");
```

```
        exit(1);
```

```
    }
```

```
    memset(&server_addr, 0, sizeof(server_addr));
```

```
    server_addr.sin_family = AF_INET;
```

```

server_addr.sin_addr.s_addr = inet_addr("10.251.236.191");
server_addr.sin_port = htons(PORT);

if (bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("Bind failed");
    close(sockfd);
    exit(1);
}

listen(sockfd, 1);
printf("Server running at 10.178.181.191:%d\n", PORT);

newsockfd = accept(sockfd, (struct sockaddr *)&client_addr, &client_len);
if (newsockfd < 0) {
    perror("Accept failed");
    close(sockfd);
    exit(1);
}

printf("Client connected from %s\n", inet_ntoa(client_addr.sin_addr));

time(&rawtime);
timeinfo = gmtime(&rawtime);
snprintf(buffer, BUFFER_SIZE,
    "%d %02d-%02d-%02d %02d:%02d:%02d UTC\n",
    (int)(rawtime / (60 * 60 * 24)),
    timeinfo->tm_mday, timeinfo->tm_mon + 1, timeinfo->tm_year % 100,
    timeinfo->tm_hour, timeinfo->tm_min, timeinfo->tm_sec);

time_t ist_time = rawtime + 19800;
struct tm *ist_timeinfo = gmtime(&ist_time);

```

```

char ist_time_str[64];
snprintf(ist_time_str, sizeof(ist_time_str),
        "%02d-%02d-%02d %02d:%02d:%02d IST\n",
        ist_timeinfo->tm_mday, ist_timeinfo->tm_mon + 1, ist_timeinfo->tm_year % 100,
        ist_timeinfo->tm_hour, ist_timeinfo->tm_min, ist_timeinfo->tm_sec);

strncat(buffer, ist_time_str, BUFFER_SIZE - strlen(buffer) - 1);

write(newsockfd, buffer, strlen(buffer));

printf("Time sent to client:\n%s", buffer);

close(newsockfd);
close(sockfd);
return 0;
}

```

Time_client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <netdb.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define SERVER_NAME "10.251.236.191"
#define PORT 8000
#define BUFFER_SIZE 128

```

```
int main() {  
    int sockfd;  
    struct sockaddr_in server_addr;  
    struct hostent *server;  
    char buffer[BUFFER_SIZE];  
    int bytes_read;  
  
    server = gethostbyname(SERVER_NAME);  
    if (server == NULL) {  
        fprintf(stderr, "ERROR: No such host\n");  
        exit(1);  
    }  
  
    sockfd = socket(AF_INET, SOCK_STREAM, 0);  
    if (sockfd < 0) {  
        perror("Socket creation failed");  
        exit(1);  
    }  
  
    memset(&server_addr, 0, sizeof(server_addr));  
    server_addr.sin_family = AF_INET;  
    server_addr.sin_port = htons(PORT);  
    memcpy(&server_addr.sin_addr.s_addr, server->h_addr, server->h_length);  
  
    if (connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {  
        perror("Connection failed");  
        close(sockfd);  
    }  
}
```

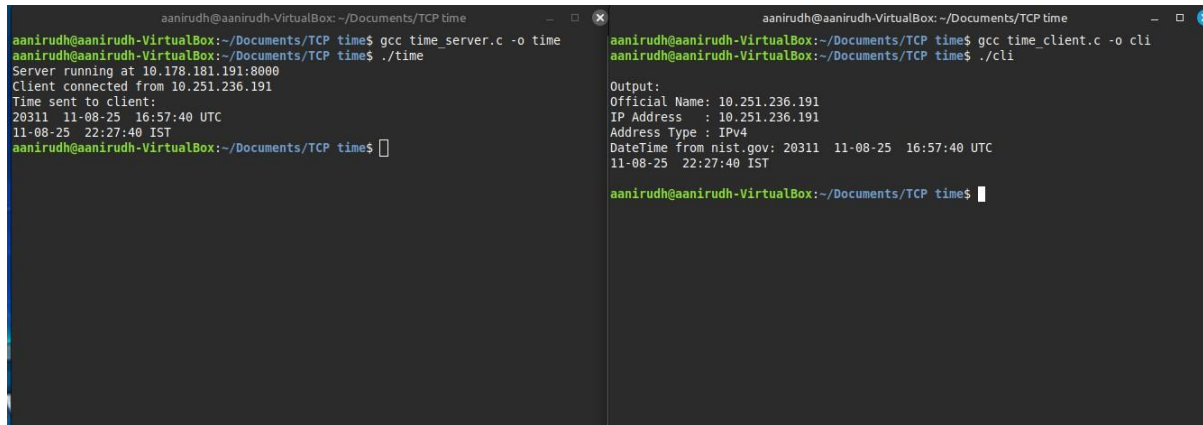


```
    exit(1);  
}
```

```
memset(buffer, 0, BUFFER_SIZE);  
bytes_read = read(sockfd, buffer, BUFFER_SIZE - 1);  
if (bytes_read < 0) {  
    perror("Read failed");  
    close(sockfd);  
    exit(1);  
}
```

```
printf("\nOutput:\n");  
printf("Official Name: %s\n", SERVER_NAME);  
printf("IP Address  : %s\n", inet_ntoa(*(struct in_addr *)server->h_addr));  
printf("Address Type : IPv4\n");  
printf("DateTime from nist.gov: %s\n", buffer);  
  
close(sockfd);  
return 0;  
}
```

OUTPUT:



```
aanirudh@aanirudh-VirtualBox: ~/Documents/TCP time
aanirudh@aanirudh-VirtualBox:~/Documents/TCP time$ gcc time_server.c -o time
aanirudh@aanirudh-VirtualBox:~/Documents/TCP time$ ./time
Server running at 10.178.181.191:8000
Client connected from 10.251.236.191
Time sent to client:
20311 11-08-25 16:57:40 UTC
11-08-25 22:27:40 IST
aanirudh@aanirudh-VirtualBox:~/Documents/TCP time$

aanirudh@aanirudh-VirtualBox: ~/Documents/TCP time
aanirudh@aanirudh-VirtualBox:~/Documents/TCP time$ gcc time_client.c -o cli
aanirudh@aanirudh-VirtualBox:~/Documents/TCP time$ ./cli
Output:
Official Name: 10.251.236.191
IP Address   : 10.251.236.191
Address Type : IPv4
DateTime from nist.gov: 20311 11-08-25 16:57:40 UTC
11-08-25 22:27:40 IST
aanirudh@aanirudh-VirtualBox:~/Documents/TCP time$
```

EX.NO: 6

TCP Concurrent Server

DATE: 02-08-2025

AIM:-

- Develop a TCP based Server application which can handle multiple clients at the same time. Test the concurrent server application by requesting a huge file.

DESCRIPTION:

Server

- Creates a TCP socket and binds it to port 8080.
- Listens for incoming client connections.
- Accepts client connections and prints client IP and port.
- Uses fork() to handle each client concurrently.
- Receives the filename requested by the client.
- Opens the file from files/ directory and sends it in chunks.
- Shuts down the write side of the socket and waits for client acknowledgment ("DONE").
- Closes file and client socket, child process exits.

Client

- Creates a TCP socket and connects to the server at 127.0.0.1:8080.
- Prompts user to enter the filename to download.
- Sends the requested filename to the server.
- Generates a unique filename locally (e.g., test.txt, test(1).txt) to avoid overwriting.
- Receives file data from the server in chunks.
- Writes data to the unique file and optionally prints content with streaming effect.
- Sends "DONE" to the server after completing the download.
- Closes the file and socket, prints completion message.

CODE:-

tcp_server.c

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <sys/socket.h>

#include <sys/types.h>

#include <signal.h>

#include <errno.h>

#include <sys/time.h>


#define PORT 8080

#define BUFFER_SIZE 4096


void handle_client(int client_sock) {

    char filename[256];

    int bytes_read;


    // Receive filename

    bytes_read = recv(client_sock, filename, sizeof(filename)-1, 0);

    if (bytes_read <= 0) {

        perror("Failed to receive filename");

        close(client_sock);

        exit(EXIT_FAILURE);

    }

    filename[bytes_read] = '\0';
```

```
printf("[Child %d] Client requested file: %s\n", getpid(), filename);
```

```
char fullpath[512];
```

```
snprintf(fullpath, sizeof(fullpath), "files/%s", filename);
```

```
FILE *fp = fopen(fullpath, "rb");
```

```
if (fp == NULL) {
```

```
    const char *msg = "ERROR:FILE_NOT_FOUND";
```

```
    send(client_sock, msg, strlen(msg), 0);
```

```
    close(client_sock);
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
printf("[Child %d] --- File transfer started ---\n", getpid());
```

```
char buffer[BUFFER_SIZE];
```

```
int n;
```

```
while ((n = fread(buffer, 1, BUFFER_SIZE, fp)) > 0) {
```

```
    if (send(client_sock, buffer, n, 0) == -1) {
```

```
        perror("Send error");
```

```
        break;
```

```
    }
```

```
}
```

```
// Close write side to signal EOF to client
```

```
shutdown(client_sock, SHUT_WR);
```

```
// Wait for client acknowledgment
```

```
char ack[16];
```

```

int ack_len = recv(client_sock, ack, sizeof(ack)-1, 0);
if (ack_len > 0) {
    ack[ack_len] = '\0';
    if (strcmp(ack, "DONE") == 0) {
        printf("[Child %d] --- File transfer complete ---\n", getpid());
        printf("--- File Content End ---\n");
    }
}

fclose(fp);
close(client_sock);
exit(EXIT_SUCCESS);
}

int main() {
    int server_sock, client_sock;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);

    signal(SIGCHLD, SIG_IGN);

    if ((server_sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    int opt = 1;
    setsockopt(server_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

```

```
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);

if (bind(server_sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
    perror("Bind failed");
    close(server_sock);
    exit(EXIT_FAILURE);
}

if (listen(server_sock, 10) < 0) {
    perror("Listen failed");
    close(server_sock);
    exit(EXIT_FAILURE);
}

printf("TCP Concurrent Server running on port %d...\n", PORT);

while (1) {
    client_sock = accept(server_sock, (struct sockaddr*)&client_addr, &addr_len);
    if (client_sock < 0) {
        if (errno == EINTR)
            continue;
        perror("Accept failed");
        continue;
    }

    printf("New client connected: %s:%d\n",
        inet_ntoa(client_addr.sin_addr),
        ntohs(client_addr.sin_port));
```

```
pid_t pid = fork();
if (pid < 0) {
    perror("Fork failed");
    close(client_sock);
} else if (pid == 0) {
    close(server_sock);
    handle_client(client_sock);
} else {
    close(client_sock);
}
}

close(server_sock);
return 0;
}
```

tcp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/stat.h>

#define PORT 8080
#define BUFFER_SIZE 1024

// Generate unique filename to avoid overwriting
```



```

void get_unique_filename(char *filename) {
    struct stat buffer;
    char new_name[256];
    int count = 1;

    if (stat(filename, &buffer) != 0) return;

    char base[200] = {0}, ext[50] = {0};
    char *dot = strrchr(filename, '.');

    if (dot && (dot - filename) < (int)sizeof(base)) {
        size_t base_len = dot - filename;
        if (base_len >= sizeof(base)) base_len = sizeof(base) - 1;
        strncpy(base, filename, base_len);
        base[base_len] = '\0';
        strncpy(ext, dot, sizeof(ext) - 1);
        ext[sizeof(ext) - 1] = '\0';
    } else {
        strncpy(base, filename, sizeof(base) - 1);
        base[sizeof(base) - 1] = '\0';
    }

    size_t max_base_len = sizeof(new_name) - strlen(ext) - 10;
    if (strlen(base) > max_base_len)
        base[max_base_len] = '\0';

    do {
        strncpy(new_name, base, sizeof(new_name) - 1);
        new_name[sizeof(new_name) - 1] = '\0';
        char suffix[32];

```

```

    snprintf(suffix, sizeof(suffix), "(%d)", count++);
    strncat(new_name, suffix, sizeof(new_name) - strlen(new_name) - 1);
    strncat(new_name, ext, sizeof(new_name) - strlen(new_name) - 1);
} while (stat(new_name, &buffer) == 0);

strncpy(filename, new_name, 255);
filename[255] = '\0';
}

int main() {
    int sock;
    struct sockaddr_in server_addr;
    char filename[256], buffer[BUFFER_SIZE];
    FILE *fp;
    ssize_t n;

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("Socket error");
        exit(1);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    if (connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Connection error");
        close(sock);
        exit(1);
    }

```

```
}
```

```
printf("Enter filename to download: ");
```

```
scanf("%255s", filename);
```

```
// Send filename to server
```

```
send(sock, filename, strlen(filename), 0);
```

```
get_unique_filename(filename);
```

```
fp = fopen(filename, "wb");
```

```
if (!fp) {
```

```
    perror("File open error");
```

```
    close(sock);
```

```
    exit(1);
```

```
}
```

```
printf("\n--- File Content (Streaming) ---\n");
```

```
// Receive file from server
```

```
while ((n = recv(sock, buffer, BUFFER_SIZE, 0)) > 0) {
```

```
    fwrite(buffer, 1, n, fp);
```

```
// Print content slowly
```

```
for (int i = 0; i < n; i++) {
```

```
    putchar(buffer[i]);
```

```
    fflush(stdout);
```

```
    usleep(30000); // 30ms delay for streaming effect
```

```
}
```

```
}
```

```
printf("\nFile received and saved as '%s'\n", filename);

printf("File transfer over, thank you for the patience\n");
```

```
const char *ack = "DONE";

send(sock, ack, strlen(ack), 0);
```

```
printf("\n--- File Content End ---\n");
```

```
fclose(fp);

close(sock);

return 0;

}
```

OUTPUT:-

```
aanirudh@aanirudh-VirtualBox: ~/Desktop/tcp_server_project/server
.c -o server
aanirudh@aanirudh-VirtualBox:~/Desktop/tcp_server_project/server$ ./server
TCP Concurrent Server running on port 8080...
New client connected: 127.0.0.1:40416
New client connected: 127.0.0.1:42418
[Child 3771] Client requested file: test.txt
[Child 3771] --- File transfer started ---
New client connected: 127.0.0.1:48740
[Child 3778] Client requested file: test.txt
[Child 3778] --- File transfer started ---
[Child 3768] Client requested file: test.txt
[Child 3768] --- File transfer started ---
[Child 3771] --- File transfer complete ---
--- File Content End ---

aanirudh@aanirudh-VirtualBox:~/Desktop/tcp_server_project/client
.c -o cli
aanirudh@aanirudh-VirtualBox:~/Desktop/tcp_server_project/client$ ./cli
Enter filename to download: test.txt

--- File Content (Streaming) ---
2WonIrS8PCXsaHBuAqTdD3S2teJwapZLOvdsGQ768H32mJHh6vFVVkoJdwISM gpBMwaZQ4950gluEY
9bItQ0dLAbeyFTjtINQw3S5B7IXEE157Iq4ZXqs02ThLfcAoVYT2Kp9RLC4RFLTEj9fbjbnZNGMu0Sjn
ZTFDwdpYEE57H0zaPGiubMeae t00AXuvbYXIwmwOKLWR9LRhy7jYl859v09FzXV6ezKkFFG7rXoMpmf
Nq4DKT6jhZUk6r ox928xmB3Aa0hWkZpYDA1ClzCCqH220TVEiawSZU5k1eoCsw48LQrx6sBFH0Wb
a oFLZvZp07K0nvt50dRkYAXE32lWJdgRk35eme0szLteD xHiXY4h10CFwLiPaK0FT1RMG7qjYKvIi
04GI2H5xK1RSZY1eB2N31LakWjEFp2WYub7HR5PN9uKamx5ueHdLkcsXqkFDGaqAxak1YP5Pp2Q0RSax
vJ0ekn7b4Lm3EaMvTc0T49JofZrShG BM87nS0bMFwNqkeypnxoChmmy86Ku7AbILF5Ee7vBZcAvwwWp
MsNchfGufos dbQlJAFShzS00mIsteAgFBrYy6Uuqnv1kvKrvCqKrku80cjHQJYhmCYhU0WHFF iDcp8
6EqITMgpX7i80buR96Cw1ECvJmk1yzZ0s0wrCVZ9S0MPgI0btKwp6RqJv8pfLhTeL30y4h4hy0Z4ZDZ
WYyqp8SuyJh0ICLwJadKYTx0oqZp2sNoDmh73800rSBjIQMRGcDr1ZmGLWehE0AUIWnhC1JKoTyXbiq
429dnJbg9xz2fG2MMzcYGI 8Smpiju2pn7FrZT940DHECS3XmLg6hEOE 2WnkLDTjWjFdeLNCV8m3JweH

File received and saved as 'test(24).txt'
File transfer over, thank you for the patience

--- File Content End ---
aanirudh@aanirudh-VirtualBox:~/Desktop/tcp_server_project/client$
```

RESULT:-

Hence the tcp has handled multiple clients at same time and transferred the file requested by them

EX NO :7

DATE : 2.8.27

TCP Concurrent Server Application

AIM:

To convert the Hangman Game Server as a concurrent Server. Test the server with multiple Clients at the same time.

DESCRIPTION:

- This application demonstrates a **TCP Concurrent Server** that allows **multiple clients** to play the Hangman game **simultaneously** using **multi-threading** or **process forking**.
- Each client connects to the server through a **TCP socket**. When a connection request is received, the server creates a **separate thread or child process** to handle that client independently. This ensures that several clients can guess words and play their own games at the same time without interfering with each other.
- The server maintains the game logic — selecting a random word, checking guesses, updating remaining chances, and sending results back to the client. The client interacts by sending letter guesses and receiving updates until the word is guessed or chances/time expire.

SOURCE CODE:

```
<Hangman_client.c>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/time.h>

#define PORT 9000
#define BUFFER_SIZE 1024
#define TIMEOUT_SECONDS 10

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <server_ip>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
}
```

```

char *server_ip = argv[1];
int sockfd;
struct sockaddr_in server_addr;
char buffer[BUFFER_SIZE];

if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}

struct timeval timeout;
timeout.tv_sec = TIMEOUT_SECONDS;
timeout.tv_usec = 0;
setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = inet_addr(server_ip);

if (connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("Connection failed");
    close(sockfd);
    exit(EXIT_FAILURE);
}

while (1) {
    int n = recv(sockfd, buffer, sizeof(buffer) - 1, 0);
    if (n <= 0) {
        printf("Server timeout or disconnected.\n");
        break;
    }
    buffer[n] = '\0';
    printf("%s", buffer);

    if (strstr(buffer, "WIN") || strstr(buffer, "Game Over")) {
        break;
    }

    char guess[10];
    fgets(guess, sizeof(guess), stdin);
    send(sockfd, guess, strlen(guess), 0);
}

close(sockfd);
return 0;

```

```
}
```

```
<Hangman_server.c>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
#include <pthread.h>
```

```
#include <ctype.h>
```

```
#include <time.h>
```

```
#define PORT 9000
```

```
#define BUFFER_SIZE 1024
```

```
#define MAX_ATTEMPTS 6
```

```
#define TIMEOUT_SECONDS 10
```

```
const char *words[] = {"computer", "network", "concurrent", "socket", "programming"};
```

```
int word_count = 5;
```

```
void *handle_client(void *arg) {
```

```
    int client_sock = *(int *)arg;
```

```
    free(arg);
```

```
    struct timeval timeout;
```

```
    timeout.tv_sec = TIMEOUT_SECONDS;
```

```
    timeout.tv_usec = 0;
```

```
    setsockopt(client_sock, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));
```

```
    const char *word = words[rand() % word_count];
```

```
    int word_len = strlen(word);
```

```
    char guessed[word_len + 1];
```

```
    for (int i = 0; i < word_len; i++) guessed[i] = '_';
```

```
    guessed[word_len] = '\0';
```

```
    int attempts = MAX_ATTEMPTS;
```

```
    char buffer[BUFFER_SIZE];
```

```
    while (attempts > 0) {
```

```
        snprintf(buffer, sizeof(buffer),
```

```
            "Word: %s Attempts left: %d (Respond in %d seconds)\nEnter a letter: ",
```

```
            guessed, attempts, TIMEOUT_SECONDS);
```



```

    send(client_sock, buffer, strlen(buffer), 0);

    int n = recv(client_sock, buffer, sizeof(buffer) - 1, 0);
    if (n <= 0) {

        snprintf(buffer, sizeof(buffer), "Timeout or disconnect! Game Over! The word was:
%s\n", word);
        send(client_sock, buffer, strlen(buffer), 0);
        break;
    }

    buffer[n] = '\0';
    char guess = tolower(buffer[0]);

    int correct = 0;
    for (int i = 0; i < word_len; i++) {
        if (word[i] == guess && guessed[i] == '_') {
            guessed[i] = guess;
            correct = 1;
        }
    }

    if (!correct) attempts--;

    if (strcmp(word, guessed) == 0) {
        snprintf(buffer, sizeof(buffer), "You WIN! The word was: %s\n", word);
        send(client_sock, buffer, strlen(buffer), 0);
        break;
    }
}

if (attempts == 0) {
    snprintf(buffer, sizeof(buffer), "Game Over! The word was: %s\n", word);
    send(client_sock, buffer, strlen(buffer), 0);
}

close(client_sock);
pthread_exit(NULL);
}

int main() {
    srand(time(NULL));

    int server_sock;
    struct sockaddr_in server_addr, client_addr;

```

```

socklen_t addr_len = sizeof(client_addr);

if ((server_sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = INADDR_ANY;

if (bind(server_sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("Bind failed");
    close(server_sock);
    exit(EXIT_FAILURE);
}

if (listen(server_sock, 5) < 0) {
    perror("Listen failed");
    close(server_sock);
    exit(EXIT_FAILURE);
}

printf("Hangman Server running on port %d...\n", PORT);

while (1) {
    int client_sock = accept(server_sock, (struct sockaddr *)&client_addr, &addr_len);
    if (client_sock < 0) {
        perror("Accept failed");
        continue;
    }

    printf("New client connected: %s\n", inet_ntoa(client_addr.sin_addr));

    int *new_sock = malloc(sizeof(int));
    *new_sock = client_sock;

    pthread_t tid;
    if (pthread_create(&tid, NULL, handle_client, new_sock) != 0) {
        perror("Thread creation failed");
        close(client_sock);
        free(new_sock);
    }
    pthread_detach(tid);
}

```

```

close(server_sock);
return 0;
}

```

OUTPUT:

```

aanirudh@aanirudh-VirtualBox: ~/Desktop/Concurrent_Hangman
aanirudh@aanirudh-VirtualBox:~/Desktop/Concurrent_Hangman$ gcc server.c -o server
aanirudh@aanirudh-VirtualBox:~/Desktop/Concurrent_Hangman$ ./server
Server listening on port 12345...
Player 100 connected
Player 101 connected
Player 102 connected

```

```

aanirudh@aanirudh-VirtualBox: ~/Desktop/Concurrent_Hangman
Word to guess: a_ma
Your ID: Player 101 Attempts left: 0
Message: Player 102 guessed 'c' - WRONG

Players:
Player Name    Attempts Left  Correct Guesses
Player 100      6              2
Player 101      0              1
Player 102      1              0

Enter one letter guess: 

```

```

aanirudh@aanirudh-VirtualBox: ~/Desktop/Concurrent_Hangman
Word to guess: a_ma
Your ID: Player 102 Attempts left: 6
Message: Player 102 guessed 'c' - WRONG

Players:
Player Name    Attempts Left  Correct Guesses
Player 100      6              2
Player 101      0              1
Player 102      1              0

Enter one letter guess: 

```

```

aanirudh@aanirudh-VirtualBox: ~/Desktop/Concurrent_Hangman
Word to guess: a_ma
Your ID: Player 102 Attempts left: 1
Message: Player 102 guessed 'c' - WRONG

Players:
Player Name    Attempts Left  Correct Guesses
Player 100      6              2
Player 101      0              1
Player 102      1              0

Enter one letter guess: 

```

RESULT:

Concurrent Hangman Server successfully handled multiple clients simultaneously

EX.NO: 8

Broadcasting

DATE: 02-08-2025

AIM:-

Broadcasting

Broadcast a message to a set of clients.

Use

SO_BROADCAST - which provides rights to send broadcast

SO_REUSEADDR allows a set of clients to bind to the same port number.

DESCRIPTION:-

Server:

- The server creates a UDP socket, enables broadcasting using SO_BROADCAST, and sends messages to the broadcast address (255.255.255.255) on a fixed port.
- It continuously accepts user input and transmits the message to all clients listening on that port.

Client:

- The client creates a UDP socket, sets SO_REUSEADDR to allow multiple clients on the same port, and binds to the broadcast port.
- It continuously listens and displays any broadcast messages received from the server.

CODE:-

Server.c

```
/* udp_broadcast_server.c */
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
#define BROADCAST_PORT 9000
```

```
#define BROADCAST_IP "255.255.255.255" // Broadcast to all in the LAN
```

```

int main() {
    int sockfd;
    struct sockaddr_in broadcast_addr;
    char message[100];

    // Create UDP socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Enable broadcast permission
    int broadcastPermission = 1;
    if (setsockopt(sockfd, SOL_SOCKET, SO_BROADCAST, &broadcastPermission,
sizeof(broadcastPermission)) < 0) {
        perror("Error enabling broadcast option");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    // Configure broadcast address
    memset(&broadcast_addr, 0, sizeof(broadcast_addr));
    broadcast_addr.sin_family = AF_INET;
    broadcast_addr.sin_port = htons(BROADCAST_PORT);
    broadcast_addr.sin_addr.s_addr = inet_addr(BROADCAST_IP);

    while (1) {
        printf("Enter message to broadcast (type 'exit' to quit): ");
        fgets(message, sizeof(message), stdin);
        message[strcspn(message, "\n")] = 0; // remove newline

        if (strcmp(message, "exit") == 0)
            break;

        // Send broadcast message
        if (sendto(sockfd, message, strlen(message), 0,
            (struct sockaddr*)&broadcast_addr, sizeof(broadcast_addr)) < 0) {
            perror("Broadcast send failed");
        }
    }
}

```

```

else {
    printf("Broadcast message sent: %s\n", message);
}
}

close(sockfd);
return 0;
}

```

Client.c

```

/* udp_broadcast_client.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define BROADCAST_PORT 9000

int main() {
    int sockfd;
    struct sockaddr_in recv_addr;
    char buffer[100];
    socklen_t addr_len = sizeof(recv_addr);

    // Create UDP socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Allow multiple clients to bind to the same port
    int reuse = 1;
    if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse)) <
0) {
        perror("Error enabling SO_REUSEADDR");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
}

```

```
// Bind the socket to the broadcast port
memset(&recv_addr, 0, sizeof(recv_addr));
recv_addr.sin_family = AF_INET;
recv_addr.sin_port = htons(BROADCAST_PORT);
recv_addr.sin_addr.s_addr = INADDR_ANY;

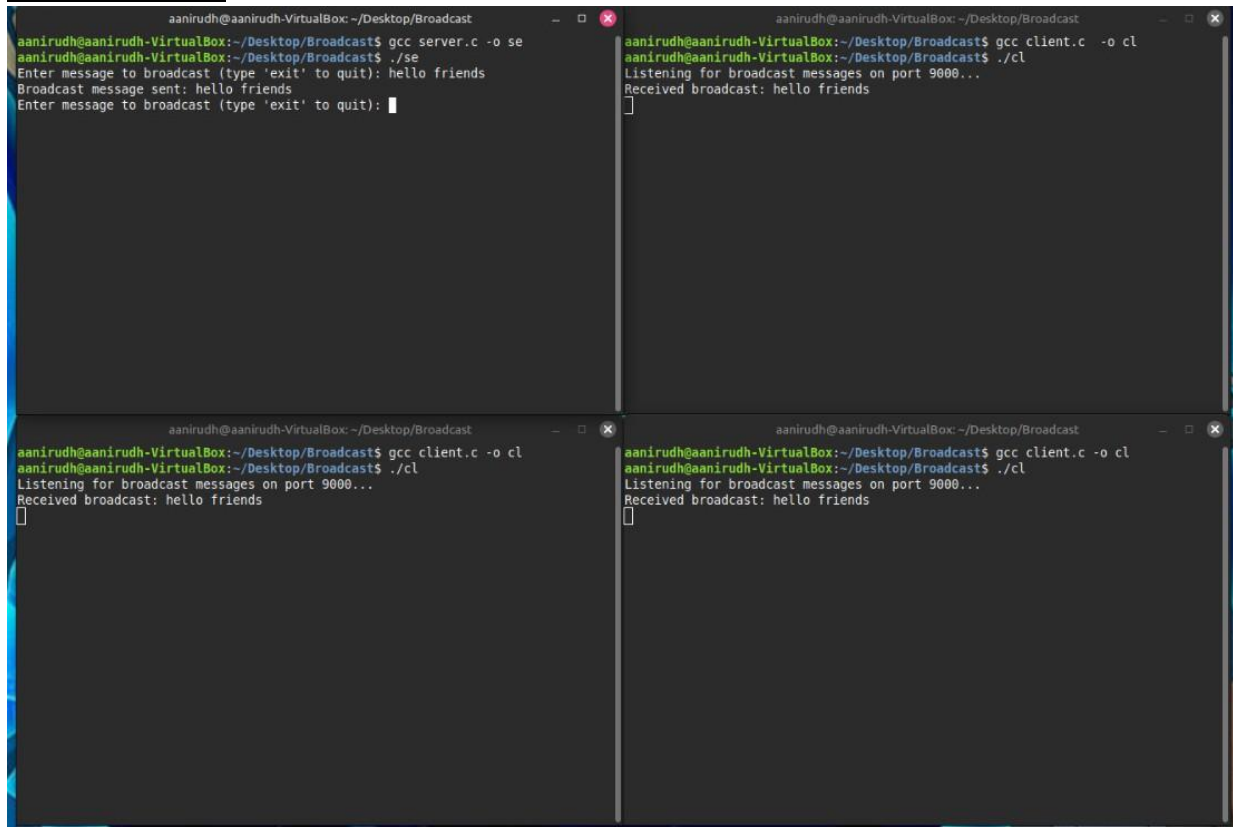
if (bind(sockfd, (struct sockaddr*)&recv_addr, sizeof(recv_addr)) < 0) {
    perror("Bind failed");
    close(sockfd);
    exit(EXIT_FAILURE);
}

printf("Listening for broadcast messages on port %d...\n", BROADCAST_PORT);

while (1) {
    memset(buffer, 0, sizeof(buffer));
    recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct sockaddr*)&recv_addr,
&addr_len);
    printf("Received broadcast: %s\n", buffer);
}

close(sockfd);
return 0;
}
```

OUTPUT:-



```
aanirudh@aanirudh-VirtualBox: ~/Desktop/Broadcast
aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast$ gcc server.c -o se
aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast$ ./se
Enter message to broadcast (type 'exit' to quit): hello friends
Broadcast message sent: hello friends
Enter message to broadcast (type 'exit' to quit):

aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast
aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast$ gcc client.c -o cl
aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast$ ./cl
Listening for broadcast messages on port 9000...
Received broadcast: hello friends

aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast
aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast$ gcc client.c -o cl
aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast$ ./cl
Listening for broadcast messages on port 9000...
Received broadcast: hello friends

aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast
aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast$ gcc client.c -o cl
aanirudh@aanirudh-VirtualBox:~/Desktop/Broadcast$ ./cl
Listening for broadcast messages on port 9000...
Received broadcast: hello friends
```

RESULT :-

Thus this program broadcasts a message to a set of clients.

EX.NO: 9

Port Scanner

DATE: 20-08-2025

AIM:-

To develop a port scanner print the open , closed and filtered ports

DESCRIPTION:-

Server:

- The server creates a UDP socket, enables broadcasting using SO_BROADCAST, and sends messages to the broadcast address (255.255.255.255) on a fixed port.
- It continuously accepts user input and transmits the message to all clients listening on that port.

Client:

- The client creates a UDP socket, sets SO_REUSEADDR to allow multiple clients on the same port, and binds to the broadcast port.
- It continuously listens and displays any broadcast messages received from the server.

CODE:-

Server.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
#include <sys/select.h>
```

```
#define MAX_PORTS 100
```

```
int main(int argc, char *argv[]) {
```

```
    if (argc < 2) {
```

```
        printf("Usage: %s <port1> <port2> ...\n", argv[0]);
```

```
        return 1;
```

```
    }
```

```
    int sockfds[MAX_PORTS], ports[MAX_PORTS], num_ports = argc - 1;
```

```
    struct sockaddr_in server;
```

```
    fd_set readfds;
```

```
    int maxfd = 0;
```

```
    // --- Create listening sockets ---
```

```
    for (int i = 0; i < num_ports; i++) {
```

```
        int port = atoi(argv[i + 1]);
```

```
        ports[i] = port;
```

```
        sockfds[i] = socket(AF_INET, SOCK_STREAM, 0);
```

```
        if (sockfds[i] < 0) { perror("socket"); exit(1); }
```

```
        int opt = 1;
```

```
        setsockopt(sockfds[i], SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

```
        memset(&server, 0, sizeof(server));
```

```
        server.sin_family = AF_INET;
```

```
server.sin_addr.s_addr = INADDR_ANY;
server.sin_port = htons(port);

if (bind(sockfds[i], (struct sockaddr*)&server, sizeof(server)) < 0) {
    perror("bind");
    close(sockfds[i]);
    sockfds[i] = -1;
    continue;
}

if (listen(sockfds[i], 5) < 0) {
    perror("listen");
    close(sockfds[i]);
    sockfds[i] = -1;
    continue;
}

printf("Server listening on port %d\n", port);
if (sockfds[i] > maxfd) maxfd = sockfds[i];
}

// --- Main loop to accept connections ---
while (1) {
    FD_ZERO(&readfds);
    for (int i = 0; i < num_ports; i++) {
        if (sockfds[i] > 0)
            FD_SET(sockfds[i], &readfds);
```

```
}

if (select(maxfd + 1, &readfds, NULL, NULL, NULL) < 0) {
    perror("select");
    break;
}

for (int i = 0; i < num_ports; i++) {
    if (sockfds[i] > 0 && FD_ISSET(sockfds[i], &readfds)) {
        int client = accept(sockfds[i], NULL, NULL);
        if (client >= 0) {
            printf("Connection accepted on port %d\n", ports[i]);
            close(client);
        }
    }
}

// --- Close sockets on exit ---
for (int i = 0; i < num_ports; i++) {
    if (sockfds[i] > 0) close(sockfds[i]);
}

return 0;
}
```

Client.c

```
/* scanner_menu.c
*
* Menu-driven TCP port scanner (single-threaded).
*
* Features:
* - Scan a range of ports on a target IP
* - Keep last scan results in memory
* - Menu options:
*   1) Perform a new scan
*   2) View Active (OPEN) ports
*   3) View Closed ports
*   4) View Filtered (BLOCKED) ports
*   5) Interactive rescan (specific port/range)
*   6) Set custom timeout (supports fractional seconds)
*   7) Save last results to a file
*   8) Exit
*
* Compile: gcc -o scanner_menu scanner_menu.c
* Run: ./scanner_menu
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```
#include <fcntl.h>
#include <errno.h>
#include <arpa/inet.h>
#include <sys/select.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/socket.h>
```

```
enum PortState { PST_UNKNOWN = 2, PST_OPEN = 1, PST_CLOSED = 0,
PST_FILTERED = -1 };
```

```
#define DEFAULT_TIMEOUT 1.0
```

```
#define MAX_HOSTLEN 256
```

```
double timeout_seconds = DEFAULT_TIMEOUT;
```

```
struct ScanResults {
    char target[MAX_HOSTLEN];
    int start_port;
    int end_port;
    int count;      /* number of ports scanned (end - start + 1) */
    enum PortState *state; /* dynamically allocated array of states */
};
```

```
/* scan single TCP port using non-blocking connect + select */
```

```
int scan_one_port(const char *ip, int port, double timeout_sec) {
```

```
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

```
    if (sockfd < 0) return -2; /* error creating socket */
```

```

/* set non-blocking */
int flags = fcntl(sockfd, F_GETFL, 0);
if (flags >= 0) fcntl(sockfd, F_SETFL, flags | O_NONBLOCK);

struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
if (inet_pton(AF_INET, ip, &addr.sin_addr) != 1) {
    close(sockfd);
    return -2;
}

int ret = connect(sockfd, (struct sockaddr*)&addr, sizeof(addr));
if (ret == 0) { close(sockfd); return 1; } /* OPEN */
else if (ret < 0 && errno != EINPROGRESS) { close(sockfd); return 0; } /*
CLOSED */

fd_set wfds;
FD_ZERO(&wfds);
FD_SET(sockfd, &wfds);

struct timeval tv;
tv.tv_sec = (int)timeout_sec;          /* integer seconds */
tv.tv_usec = (timeout_sec - tv.tv_sec) * 1000000; /* fractional part in µs */

ret = select(sockfd + 1, NULL, &wfds, NULL, &tv);

```

```

if (ret == 1) {
    int so_error = 0;
    socklen_t len = sizeof(so_error);
    if (getsockopt(sockfd, SOL_SOCKET, SO_ERROR, &so_error, &len) < 0) {
close(sockfd); return 0; }
    close(sockfd);
    if (so_error == 0) return 1;
    else return 0;
} else if (ret == 0) { close(sockfd); return -1; } /* FILTERED */
else { close(sockfd); return 0; }
}

```

```

/* allocate a ScanResults object for a target and port range */
struct ScanResults *alloc_results(const char *target, int start, int end) {
    if (!target || start < 0 || end < start) return NULL;
    struct ScanResults *res = malloc(sizeof(*res));
    if (!res) return NULL;
    strncpy(res->target, target, MAX_HOSTLEN-1);
    res->target[MAX_HOSTLEN-1] = '\0';
    res->start_port = start;
    res->end_port = end;
    res->count = end - start + 1;
    res->state = calloc(res->count, sizeof(enum PortState));
    if (!res->state) { free(res); return NULL; }
    for (int i = 0; i < res->count; ++i) res->state[i] = PST_UNKNOWN;
    return res;
}

```



```
void free_results(struct ScanResults *r) {  
    if (!r) return;  
    free(r->state);  
    free(r);  
}
```

```
/* perform a full scan and populate results */
```

```
void perform_scan(struct ScanResults *r) {  
    if (!r) return;  
    printf("Scanning %s ports %d-%d with timeout %.3f second(s)...\n",  
        r->target, r->start_port, r->end_port, timeout_seconds);  
    for (int p = r->start_port; p <= r->end_port; ++p) {  
        int s = scan_one_port(r->target, p, timeout_seconds);  
        int idx = p - r->start_port;  
        if (s == 1) r->state[idx] = PST_OPEN;  
        else if (s == 0) r->state[idx] = PST_CLOSED;  
        else if (s == -1) r->state[idx] = PST_FILTERED;  
        else r->state[idx] = PST_UNKNOWN;  
        /* optional small progress print per N ports */  
        if ((p - r->start_port) % 50 == 0) fflush(stdout);  
    }  
    printf("Scan finished.\n");  
}
```

```
/* print ports by state */
```

```
void print_by_state(struct ScanResults *r, enum PortState want) {  
    if (!r) { printf("No scan results available. Perform a scan first.\n"); return; }
```

```

int found = 0;
for (int i = 0; i < r->count; ++i) {
    if (r->state[i] == want) {
        int port = r->start_port + i;
        const char *name = (want == PST_OPEN) ? "OPEN" :
            (want == PST_CLOSED) ? "CLOSED" :
            (want == PST_FILTERED) ? "FILTERED" : "UNKNOWN";
        printf("Port %d: %s\n", port, name);
        found = 1;
    }
}

if (!found) {
    if (want == PST_OPEN) printf("No OPEN ports found in the last scan.\n");
    else if (want == PST_CLOSED) printf("No CLOSED ports found in the last
scan.\n");
    else if (want == PST_FILTERED) printf("No FILTERED ports found in the last
scan.\n");
    else printf("No ports with requested state.\n");
}

/* interactive rescan of a single port or small range */
void interactive_rescan(struct ScanResults *r) {
    if (!r) { printf("No scan results available to rescan. Perform a scan first.\n");
return; }

    char line[128];
    printf("Enter port or range to rescan (e.g. 80 or 100-110): ");
    if (!fgets(line, sizeof(line), stdin)) return;

```

```

int a = 0, b = 0;
if (sscanf(line, "%d-%d", &a, &b) == 2) {
    /* range */
} else if (sscanf(line, "%d", &a) == 1) {
    b = a;
} else {
    printf("Invalid input.\n");
    return;
}
if (a < r->start_port || b > r->end_port || a > b) {
    printf("Range must be within last scanned bounds: %d-%d\n", r->start_port,
r->end_port);
    return;
}
printf("Rescanning %d-%d...\n", a, b);
for (int p = a; p <= b; ++p) {
    int s = scan_one_port(r->target, p, timeout_seconds);
    int idx = p - r->start_port;
    if (s == 1) r->state[idx] = PST_OPEN;
    else if (s == 0) r->state[idx] = PST_CLOSED;
    else if (s == -1) r->state[idx] = PST_FILTERED;
    else r->state[idx] = PST_UNKNOWN;
    printf("Port %d: %s\n", p,
        (r->state[idx]==PST_OPEN) ? "OPEN" :
        (r->state[idx]==PST_CLOSED) ? "CLOSED" :
        (r->state[idx]==PST_FILTERED) ? "FILTERED" : "UNKNOWN");
}
printf("Rescan complete.\n");

```

```
}
```

```
/* save results to a file */
```

```
void save_results(struct ScanResults *r) {
```

```
    if (!r) { printf("No results to save.\n"); return; }
```

```
    char fname[256];
```

```
    printf("Enter filename to save results: ");
```

```
    if (!fgets(fname, sizeof(fname), stdin)) return;
```

```
    fname[strcspn(fname, "\r\n")] = '\0';
```

```
    if (strlen(fname) == 0) { printf("No filename entered.\n"); return; }
```

```
    FILE *f = fopen(fname, "w");
```

```
    if (!f) { perror("fopen"); return; }
```

```
    fprintf(f, "Scan results for %s ports %d-%d (timeout=%.3f s)\n\n",
```

```
           r->target, r->start_port, r->end_port, timeout_seconds);
```

```
    for (int i = 0; i < r->count; ++i) {
```

```
        int port = r->start_port + i;
```

```
        const char *name = (r->state[i] == PST_OPEN) ? "OPEN" :
```

```
                           (r->state[i] == PST_CLOSED) ? "CLOSED" :
```

```
                           (r->state[i] == PST_FILTERED) ? "FILTERED" : "UNKNOWN";
```

```
        fprintf(f, "Port %d: %s\n", port, name);
```

```
    }
```

```
    fclose(f);
```

```
    printf("Results saved to %s\n", fname);
```

```
}
```

```
/* read a line of input safely */
```

```
static void read_line(char *buf, size_t n) {  
    if (!fgets(buf, n, stdin)) { buf[0] = '\0'; return; }  
    buf[strcspn(buf, "\r\n")] = '\0';  
}
```

```
int main() {  
    struct ScanResults *results = NULL;  
    char input[256];  
  
    while (1) {  
        printf("\n=== TCP Port Scanner Menu ===\n");  
        printf("1) Perform new scan\n");  
        printf("2) View Active (OPEN) ports\n");  
        printf("3) View Closed ports\n");  
        printf("4) View Filtered (BLOCKED) ports\n");  
        printf("5) Interactive rescan (specific port/range)\n");  
        printf("6) Set custom timeout (current = %.3f sec)\n", timeout_seconds);  
        printf("7) Save last results to file\n");  
        printf("8) Exit\n");  
        printf("Choose an option: ");  
        read_line(input, sizeof(input));  
        int opt = atoi(input);  
  
        if (opt == 1) {  
            char ip[MAX_HOSTLEN];  
            int start, end;  
            printf("Enter target IP (IPv4): ");
```

```
read_line(ip, sizeof(ip));
if (strlen(ip) == 0) { printf("Invalid IP.\n"); continue; }
printf("Enter start port: ");
read_line(input, sizeof(input));
start = atoi(input);
printf("Enter end port: ");
read_line(input, sizeof(input));
end = atoi(input);
if (start <= 0 || end <= 0 || end < start || end > 65535) {
    printf("Invalid port range.\n");
    continue;
}
if (results) free_results(results);
results = alloc_results(ip, start, end);
if (!results) { printf("Failed to allocate results.\n"); continue; }
perform_scan(results);
} else if (opt == 2) { print_by_state(results, PST_OPEN); }
else if (opt == 3) { print_by_state(results, PST_CLOSED); }
else if (opt == 4) { print_by_state(results, PST_FILTERED); }
else if (opt == 5) { interactive_rescan(results); }
else if (opt == 6) {
    printf("Enter timeout in seconds (current %.3f): ", timeout_seconds);
    read_line(input, sizeof(input));
    double t = atof(input);
    if (t <= 0.0) printf("Timeout must be > 0.\n");
    else {
        timeout_seconds = t;
    }
}
```

```

        printf("Timeout set to %.3f second(s).\n", timeout_seconds);
    }

    } else if (opt == 7) { save_results(results); }

    else if (opt == 8) { printf("Exiting.\n"); break; }

    else { printf("Invalid option. Try again.\n"); }

}

if (results) free_results(results);

return 0;

}

```

OUTPUT:-

```

aanirudh@aanirudh-VirtualBox: ~/Desktop/Port
aanirudh@aanirudh-VirtualBox:~/Desktop/Port$ gcc server.c -o s
aanirudh@aanirudh-VirtualBox:~/Desktop/Port$ ./s 8091 9011
Server listening on port 8091
Server listening on port 9011
^C
aanirudh@aanirudh-VirtualBox:~/Desktop/Port$ ls
c  cl  cli  client  client.c  file.txt  s  se  ser  server  server.c
aanirudh@aanirudh-VirtualBox:~/Desktop/Port$ ./s 8080 8081 8082 8090
Server listening on port 8080
Server listening on port 8081
Server listening on port 8082
Server listening on port 8090
Connection accepted on port 8080
Connection accepted on port 8081
Connection accepted on port 8082
Connection accepted on port 8090
Connection accepted on port 8080
Connection accepted on port 8081
Connection accepted on port 8082
Connection accepted on port 8090
[

aanirudh@aanirudh-VirtualBox:~/Desktop/Port$ ./cl
==== TCP Port Scanner Menu ====
1) Perform new scan
2) View Active (OPEN) ports
3) View Closed ports
4) View Filtered (BLOCKED) ports
5) Interactive rescan (specific port/range)
6) Set custom timeout (current = 1.000 sec)
7) Save last results to file
8) Exit
Choose an option: 3
No scan results available. Perform a scan first.

==== TCP Port Scanner Menu ====
1) Perform new scan
2) View Active (OPEN) ports
3) View Closed ports
4) View Filtered (BLOCKED) ports
5) Interactive rescan (specific port/range)
6) Set custom timeout (current = 1.000 sec)
7) Save last results to file
8) Exit
Choose an option: [

aanirudh@aanirudh-VirtualBox:~/Desktop/Port$
2) View Active (OPEN) ports
3) View Closed ports
4) View Filtered (BLOCKED) ports
5) Interactive rescan (specific port/range)
6) Set custom timeout (current = 1.000 sec)
7) Save last results to file
8) Exit
Choose an option: 1
Enter target IP (IPv4): 127.0.0.1
Enter start port: 8079
Enter end port: 8092
Scanning 127.0.0.1 ports 8079-8092 with timeout 1.000 second(s)...
Scan finished.

==== TCP Port Scanner Menu ====
1) Perform new scan
2) View Active (OPEN) ports
3) View Closed ports
4) View Filtered (BLOCKED) ports
5) Interactive rescan (specific port/range)
6) Set custom timeout (current = 1.000 sec)
7) Save last results to file
8) Exit
Choose an option: [

aanirudh@aanirudh-VirtualBox:~/Desktop/Port$
Scan finished.

==== TCP Port Scanner Menu ====
1) Perform new scan
2) View Active (OPEN) ports
3) View Closed ports
4) View Filtered (BLOCKED) ports
5) Interactive rescan (specific port/range)
6) Set custom timeout (current = 1.000 sec)
7) Save last results to file
8) Exit
Choose an option: 4
Port 8080: FILTERED

==== TCP Port Scanner Menu ====
1) Perform new scan
2) View Active (OPEN) ports
3) View Closed ports
4) View Filtered (BLOCKED) ports
5) Interactive rescan (specific port/range)
6) Set custom timeout (current = 1.000 sec)
7) Save last results to file
8) Exit
Choose an option: [

```

RESULT:-

Thus the program scans the port for open , closed and filtered.

EX.NO: 10

Simulation of DHCP Server

DATE: 20-08-2025

AIM:-

To write a code to simulate a DHCP server and also write a client application to test the service offered by the server.

A DHCP Server is a network server that automatically provides and assigns IP addresses, default gateways and other network parameters to client devices. This exercise involves assigning IP addresses alone.

DESCRIPTION:-

Server (server.c)

- Create a UDP socket and bind it to port 6767.
- Initialize a pool of IP addresses and mark all as free.
- Continuously listen for client messages like DISCOVER, REQUEST, RENEW, REBIND, and RELEASE.
- Assign IPs from the pool, renew leases, or release IPs based on client requests.
- Run a separate thread to check for expired leases and free IPs automatically.

Client (client.c)

- Create a UDP socket to communicate with the server at 127.0.0.1:6767.
- Send a DISCOVER message and wait for an OFFER from the server.
- Send REQUEST for the offered IP and wait for ACK confirming the lease.
- Start a lease manager thread that sends RENEW, REBIND, or RELEASE based on lease timing.
- Continuously listen for server responses to maintain or release the lease automatically.

CODE:-

Server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```
#include <arpa/inet.h>
#include <pthread.h>
#include <time.h>
```

```
#define SERVER_PORT 6767
#define MAX_BUFFER 1024
#define MAX_IPS 10
#define LEASE_TIME 20
```

```
// IP pool
char *ip_pool[MAX_IPS] = {
    "192.168.1.100",
    "192.168.1.101",
    "192.168.1.102",
    "192.168.1.103",
    "192.168.1.104",
    "192.168.1.105",
    "192.168.1.106",
    "192.168.1.107",
    "192.168.1.108",
    "192.168.1.109"
};
```

```
// Lease structure
typedef struct {
    char ip[50];
    struct sockaddr_in client_addr;
    time_t lease_start;
    time_t last_seen; // updated on each REQUEST / RENEW / REBIND
    int active;       // 0 = free, 1 = assigned
} Lease;
```

```
Lease leases[MAX_IPS];
pthread_mutex_t lease_lock = PTHREAD_MUTEX_INITIALIZER;
// Monitor thread to expire inactive clients
void *lease_monitor(void *arg) {
    while (1) {
        sleep(1);
        pthread_mutex_lock(&lease_lock);
        time_t now = time(NULL);
        for (int i = 0; i < MAX_IPS; i++) {
            if (leases[i].active && difftime(now, leases[i].last_seen) >= LEASE_TIME) {
```

```

        printf("[SERVER] Client %s:%d inactive. IP %s released and returned to
pool.\n",
            inet_ntoa(leases[i].client_addr.sin_addr),
            ntohs(leases[i].client_addr.sin_port),
            leases[i].ip);
        leases[i].active = 0;
    }
}
pthread_mutex_unlock(&lease_lock);
}
return NULL;
}

```

```

int main() {
    int sockfd;
    char buffer[MAX_BUFFER];
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);

    for (int i = 0; i < MAX_IPS; i++) leases[i].active = 0;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(SERVER_PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("[SERVER] Listening on port %d...\n", SERVER_PORT);

    pthread_t monitor_thread;
    pthread_create(&monitor_thread, NULL, lease_monitor, NULL);
}

```

```

while (1) {
    memset(buffer, 0, MAX_BUFFER);
    int n = recvfrom(sockfd, buffer, MAX_BUFFER, 0,
                    (struct sockaddr *)&client_addr, &addr_len);
    if (n <= 0) continue;
    buffer[n] = '\0';

    pthread_mutex_lock(&lease_lock);

    if (strncmp(buffer, "DISCOVER", 8) == 0) {
        int assigned = -1;
        for (int i = 0; i < MAX_IPS; i++) {
            if (!leases[i].active) {
                assigned = i;
                break;
            }
        }
        if (assigned == -1) {
            char msg[] = "NO_IPS_AVAILABLE";
            sendto(sockfd, msg, strlen(msg), 0,
                    (struct sockaddr *)&client_addr, addr_len);
            pthread_mutex_unlock(&lease_lock);
            continue;
        }
        char offer[MAX_BUFFER];
        snprintf(offer, sizeof(offer), "OFFER %s", ip_pool[assigned]);
        sendto(sockfd, offer, strlen(offer), 0,
                (struct sockaddr *)&client_addr, addr_len);
        printf("[SERVER] Sent OFFER %s\n", ip_pool[assigned]);
    }

    else if (strncmp(buffer, "REQUEST", 7) == 0) {
        char requested_ip[50];
        sscanf(buffer, "REQUEST %s", requested_ip);
        for (int i = 0; i < MAX_IPS; i++) {
            if (strcmp(ip_pool[i], requested_ip) == 0) {
                leases[i].active = 1;
                leases[i].lease_start = leases[i].last_seen = time(NULL);

                leases[i].client_addr = client_addr;
                strcpy(leases[i].ip, requested_ip);
            }
        }
    }
}

```

```

        char ack[MAX_BUFFER];
        snprintf(ack, sizeof(ack), "ACK %s %d", requested_ip, LEASE_TIME);
        sendto(sockfd, ack, strlen(ack), 0,
            (struct sockaddr *)&client_addr, addr_len);
        printf("[SERVER] Assigned %s to %s:%d (lease reset to %d sec)\n",
            requested_ip,
            inet_ntoa(client_addr.sin_addr),
            ntohs(client_addr.sin_port),
            LEASE_TIME);
    }
}
} else if (strncmp(buffer, "RENEW", 5) == 0) {
    char renew_ip[50];
    sscanf(buffer, "RENEW %s", renew_ip);
    for (int i = 0; i < MAX_IPS; i++) {
        if (leases[i].active && strcmp(leases[i].ip, renew_ip) == 0) {
            leases[i].lease_start = leases[i].last_seen = time(NULL);
            char renew_ack[MAX_BUFFER];
            snprintf(renew_ack, sizeof(renew_ack), "RENEW_ACK %s %d",
renew_ip, LEASE_TIME);
            sendto(sockfd, renew_ack, strlen(renew_ack), 0,
                (struct sockaddr *)&client_addr, addr_len);
            printf("[SERVER] Lease renewed for %s (reset to %d sec)\n",
                renew_ip, LEASE_TIME);
        }
    }
}
} else if (strncmp(buffer, "REBIND", 6) == 0) {
    char rebind_ip[50];
    sscanf(buffer, "REBIND %s", rebind_ip);
    for (int i = 0; i < MAX_IPS; i++) {
        if (leases[i].active && strcmp(leases[i].ip, rebind_ip) == 0) {
            leases[i].lease_start = leases[i].last_seen = time(NULL);
            char rebind_ack[MAX_BUFFER];
            snprintf(rebind_ack, sizeof(rebind_ack), "REBIND_ACK %s %d",
rebind_ip, LEASE_TIME);
            sendto(sockfd, rebind_ack, strlen(rebind_ack), 0,
                (struct sockaddr *)&client_addr, addr_len);

            printf("[SERVER] Lease rebinded for %s (reset to %d sec)\n",
                rebind_ip, LEASE_TIME);
        }
    }
}
}

```

```

    }
}

else if (strncmp(buffer, "RELEASE", 7) == 0) {
    char released_ip[50];
    sscanf(buffer, "RELEASE %s", released_ip);
    for (int i = 0; i < MAX_IPS; i++) {
        if (leases[i].active && strcmp(leases[i].ip, released_ip) == 0) {
            leases[i].active = 0;
            printf("[SERVER] IP %s released by client %s:%d and returned to
pool\n",
                released_ip,
                inet_ntoa(client_addr.sin_addr),
                ntohs(client_addr.sin_port));
        }
    }
}

pthread_mutex_unlock(&lease_lock);
}

close(sockfd);
return 0;
}

```

Client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#include <time.h>
#define SERVER_PORT 6767
#define MAX_BUFFER 1024
#define LEASE_TIME 20

int sockfd;
struct sockaddr_in server_addr;

```

```

socklen_t addr_len;
char my_ip[50] = "";
int lease_acquired = 0;

pthread_mutex_t lease_lock = PTHREAD_MUTEX_INITIALIZER;

// Thread to manage lease, renew, rebind, and release
void *lease_manager(void *arg) {
    double elapsed = 0.0;

    while (1) {
        usleep(500000); // 0.5 sec
        if (!lease_acquired) continue;

        elapsed += 0.5;

        // Send RENEW at 10 sec
        if (elapsed >= 10.0 && elapsed < 10.5) {
            char msg[MAX_BUFFER];
            snprintf(msg, sizeof(msg), "RENEW %s", my_ip);
            sendto(sockfd, msg, strlen(msg), 0,
                (struct sockaddr *)&server_addr, addr_len);
            printf("[CLIENT] Sending RENEW request for %s...\n", my_ip);
        }

        // Send REBIND at 17.5 sec if no successful RENEW
        if (elapsed >= 17.5 && elapsed < 18.0) {
            char msg[MAX_BUFFER];
            snprintf(msg, sizeof(msg), "REBIND %s", my_ip);
            sendto(sockfd, msg, strlen(msg), 0,
                (struct sockaddr *)&server_addr, addr_len);
            printf("[CLIENT] No ACK for renew. Sending REBIND request for %s...\n",
my_ip);
        }

        // Lease expires at 20 sec
        if (elapsed >= LEASE_TIME) {
            char msg[MAX_BUFFER];
            snprintf(msg, sizeof(msg), "RELEASE %s", my_ip);

            sendto(sockfd, msg, strlen(msg), 0,
                (struct sockaddr *)&server_addr, addr_len);

```

```

    printf("[CLIENT] Lease expired. Releasing %s back to pool.\n", my_ip);
    printf("[CLIENT] RELEASE sent for %s.\n", my_ip);

    pthread_mutex_lock(&lease_lock);
    lease_acquired = 0;
    my_ip[0] = '\0';
    pthread_mutex_unlock(&lease_lock);

    elapsed = 0.0; // reset timer for next lease
}
}

return NULL;
}

// Function to handle receiving ACK or RENEW_ACK
void handle_server_response(char *buffer) {
    pthread_mutex_lock(&lease_lock);

    if (strncmp(buffer, "ACK", 3) == 0 || strncmp(buffer, "RENEW_ACK", 9) == 0 ||
        strncmp(buffer, "REBIND_ACK", 10) == 0) {
        char ip[50];
        sscanf(buffer, "%*s %s %*d", ip);
        strcpy(my_ip, ip);
        lease_acquired = 1;
        printf("[CLIENT] Lease renewed/acquired: %s (timer reset to %d sec)\n",
            my_ip, LEASE_TIME);
    }

    pthread_mutex_unlock(&lease_lock);
}

int main() {
    char buffer[MAX_BUFFER];
    addr_len = sizeof(server_addr);

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

```



```

memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(SERVER_PORT);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

pthread_t lease_thread;
pthread_create(&lease_thread, NULL, lease_manager, NULL);

// Initial DISCOVER
char discover[] = "DISCOVER";
sendto(sockfd, discover, strlen(discover), 0,
      (struct sockaddr *)&server_addr, addr_len);
printf("[CLIENT] Sending DISCOVER...\n");

int n = recvfrom(sockfd, buffer, MAX_BUFFER, 0,
      (struct sockaddr *)&server_addr, &addr_len);
buffer[n] = '\0';
printf("[CLIENT] Received: %s\n", buffer);

if (strncmp(buffer, "OFFER", 5) == 0) {
    char offered_ip[50];
    sscanf(buffer, "OFFER %s", offered_ip);

    // Send REQUEST
    char request[MAX_BUFFER];
    snprintf(request, sizeof(request), "REQUEST %s", offered_ip);
    sendto(sockfd, request, strlen(request), 0,
      (struct sockaddr *)&server_addr, addr_len);
    printf("[CLIENT] Sending REQUEST for %s...\n", offered_ip);

    n = recvfrom(sockfd, buffer, MAX_BUFFER, 0,
      (struct sockaddr *)&server_addr, &addr_len);
    buffer[n] = '\0';
    printf("[CLIENT] Received: %s\n", buffer);

    if (strncmp(buffer, "ACK", 3) == 0) {
        handle_server_response(buffer);
    }
} else {
    printf("[CLIENT] No IP available from server.\n");
}
}

```

```
// Keep listening for server responses (ACKs)
while (1) {
    n = recvfrom(sockfd, buffer, MAX_BUFFER, 0,
        (struct sockaddr *)&server_addr, &addr_len);
    buffer[n] = '\0';
    handle_server_response(buffer);
}

close(sockfd);
return 0;
}
```

OUTPUT:-

```
aanirudh@aanirudh-VirtualBox: ~/Desktop/DHCP
aanirudh@aanirudh-VirtualBox:~/Desktop/DHCP$ ls
cl cll client.c clj cll clo se server.c
aanirudh@aanirudh-VirtualBox:~/Desktop/DHCP$ gcc server.c -o se
aanirudh@aanirudh-VirtualBox:~/Desktop/DHCP$ ./se
[SERVER] Listening on port 6767...
[SERVER] Sent OFFER 192.168.1.100
[SERVER] Assigned 192.168.1.100 to 127.0.0.1:37472 (lease reset to 20 sec)
[SERVER] Sent OFFER 192.168.1.101
[SERVER] Assigned 192.168.1.101 to 127.0.0.1:42091 (lease reset to 20 sec)
[SERVER] Sent OFFER 192.168.1.102
[SERVER] Assigned 192.168.1.102 to 127.0.0.1:51878 (lease reset to 20 sec)
[SERVER] Lease renewed for 192.168.1.100 (reset to 20 sec)
[SERVER] Lease renewed for 192.168.1.101 (reset to 20 sec)
[SERVER] Lease renewed for 192.168.1.102 (reset to 20 sec)
[SERVER] Lease rebinding for 192.168.1.100 (reset to 20 sec)
[SERVER] IP 192.168.1.100 released by client 127.0.0.1:37472 and returned to pool
[SERVER] Lease rebinding for 192.168.1.101 (reset to 20 sec)
[SERVER] Lease rebinding for 192.168.1.102 (reset to 20 sec)

aanirudh@aanirudh-VirtualBox:~/Desktop/DHCP$ gcc client.c -o cl
aanirudh@aanirudh-VirtualBox:~/Desktop/DHCP$ ./cl
[CLIENT] Sending DISCOVER...
[CLIENT] Received: OFFER 192.168.1.101
[CLIENT] Sending REQUEST for 192.168.1.101...
[CLIENT] Received: ACK 192.168.1.101 20
[CLIENT] Lease renewed/acquired: 192.168.1.101 (timer reset to 20 sec)
[CLIENT] Sending RENEW request for 192.168.1.101...
[CLIENT] Lease renewed/acquired: 192.168.1.101 (timer reset to 20 sec)
[CLIENT] No ACK for renew, Sending REBIND request for 192.168.1.101...
[CLIENT] Lease renewed/acquired: 192.168.1.101 (timer reset to 20 sec)

aanirudh@aanirudh-VirtualBox:~/Desktop/DHCP$ gcc client.c -o cl
aanirudh@aanirudh-VirtualBox:~/Desktop/DHCP$ ./cl
[CLIENT] Sending DISCOVER...
[CLIENT] Received: OFFER 192.168.1.100
[CLIENT] Sending REQUEST for 192.168.1.100...
[CLIENT] Received: ACK 192.168.1.100 20
[CLIENT] Lease renewed/acquired: 192.168.1.100 (timer reset to 20 sec)
[CLIENT] Sending RENEW request for 192.168.1.100...
[CLIENT] Lease renewed/acquired: 192.168.1.100 (timer reset to 20 sec)
[CLIENT] No ACK for renew, Sending REBIND request for 192.168.1.100...
[CLIENT] Lease renewed/acquired: 192.168.1.100 (timer reset to 20 sec)
[CLIENT] Lease expired, Releasing 192.168.1.100 back to pool.
[CLIENT] RELEASE sent for 192.168.1.100.

aanirudh@aanirudh-VirtualBox:~/Desktop/DHCP$ gcc client.c -o cl
aanirudh@aanirudh-VirtualBox:~/Desktop/DHCP$ ./cl
[CLIENT] Sending DISCOVER...
[CLIENT] Received: OFFER 192.168.1.102
[CLIENT] Sending REQUEST for 192.168.1.102...
[CLIENT] Received: ACK 192.168.1.102 20
[CLIENT] Lease renewed/acquired: 192.168.1.102 (timer reset to 20 sec)
[CLIENT] Sending RENEW request for 192.168.1.102...
[CLIENT] Lease renewed/acquired: 192.168.1.102 (timer reset to 20 sec)
[CLIENT] No ACK for renew, Sending REBIND request for 192.168.1.102...
[CLIENT] Lease renewed/acquired: 192.168.1.102 (timer reset to 20 sec)
```

RESULT:-

Thus, the program simulates a DHCP server that assigns IP addresses automatically to clients.

The client application tests the service by sending DISCOVER, requesting an IP, and handling lease renewal and release.

EX.NO: 11

MULTI CASTING

DATE: 08-09-2025

AIM:-

Develop an application to send a message to a Group.

DESCRIPTION:-

1. server.c (Multicast Sender)

Creates a UDP socket.

Prepares a multicast address (239.0.0.1:12345).

Reads messages from the keyboard.

Sends messages to all clients subscribed to the multicast group.

Behavior: Any message typed appears to all clients listening on the multicast IP and port.

2. client.c (UDP Client without joining multicast)

Creates a UDP socket and binds to port 12345.

Does not join the multicast group, so it cannot receive multicast messages.

Prints a confirmation that the client is "connected."

Behavior: Shows that a client can bind to the port, but won't get multicast messages unless it joins the group.

3. client2.c (UDP Client intended to receive server messages)

Creates a UDP socket.

Sets up server address (239.0.0.1:12345).

Optionally receives a single message from the server.

Behavior: Attempts to receive one server message; multicast group code is commented out, so it may not work as intended.

CODE:-

Server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

#define MULTICAST_IP "239.0.0.1"
#define PORT 12345

int main() {
    int sockfd;
    struct sockaddr_in multicastAddr;
    char message[256];

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&multicastAddr, 0, sizeof(multicastAddr));
    multicastAddr.sin_family = AF_INET;
    multicastAddr.sin_addr.s_addr = inet_addr(MULTICAST_IP);
    multicastAddr.sin_port = htons(PORT);

    printf("● Multicast Server started. Type messages to send:\n");

    while (1) {
        printf("> ");
        fflush(stdout);
```

```

        if (fgets(message, sizeof(message), stdin) == NULL)
            break;
        sendto(sockfd, message, strlen(message), 0,
            (struct sockaddr*)&multicastAddr, sizeof(multicastAddr));
    }

```

```

close(sockfd);
return 0;
}

```

Client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

```

```

#define MULTICAST_IP "239.0.0.1"
#define PORT 12345

```

```

int main() {
    int sockfd;
    struct sockaddr_in localAddr;
    struct ip_mreq group;
    char buffer[256];
    int n;

```

```

    // Create UDP socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

```

```

    // Allow multiple clients to use the same port
    int reuse = 1;
    if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, (char*)&reuse,
        sizeof(reuse)) < 0) {
        perror("Setting SO_REUSEADDR failed");
        close(sockfd);
    }

```

```

    exit(EXIT_FAILURE);
}

// Bind to the multicast port
memset(&localAddr, 0, sizeof(localAddr));
localAddr.sin_family = AF_INET;
localAddr.sin_addr.s_addr = htonl(INADDR_ANY);
localAddr.sin_port = htons(PORT);

if (bind(sockfd, (struct sockaddr*)&localAddr, sizeof(localAddr)) < 0) {
    perror("Bind failed");
    close(sockfd);
    exit(EXIT_FAILURE);
}

/*
group.imr_multiaddr.s_addr = inet_addr(MULTICAST_IP);
group.imr_interface.s_addr = htonl(INADDR_ANY);

if (setsockopt(sockfd, IPPROTO_IP, IP_ADD_MEMBERSHIP, (char*)&group,
sizeof(group)) < 0) {
    perror("Joining multicast group failed");
    close(sockfd);
    exit(EXIT_FAILURE);
}
*/

printf("✅ Client connected (without joining multicast group) on port %d\n",
PORT);

// Receive messages
while (1) {
    memset(buffer, 0, sizeof(buffer));
    n = recv(sockfd, buffer, sizeof(buffer) - 1, 0);
    if (n < 0) {
        perror("recv failed");
        break;
    }

    buffer[n] = '\0'; // Null-terminate the received string
    printf("📧 Received message: %s\n", buffer);
}

```

```
}

// Leave group and close socket (COMMENTED OUT since not joined)
/*
if (setsockopt(sockfd, IPPROTO_IP, IP_DROP_MEMBERSHIP, (char*)&group,
sizeof(group)) < 0) {
    perror("Dropping multicast group failed");
}
*/

close(sockfd);
return 0;
}
```

OUTPUT:-

```
aanirudh@aanirudh-VirtualBox: ~/Desktop/Group
aanirudh@aanirudh-VirtualBox:~/Desktop/Group$ ./cc
✓ Client joined multicast group 239.0.0.1:12345
Received: hello group
Received: i am in group 00

aanirudh@aanirudh-VirtualBox:~/Desktop/Group$ ./c11
✓ Client connected (without joining multicast group) on port 12345
Received message: 1 am in group 00

aanirudh@aanirudh-VirtualBox:~/Desktop/Group$ gcc server.c -o ss
aanirudh@aanirudh-VirtualBox:~/Desktop/Group$ ./ss
● Multicast Server started. Type messages to send:
> hello group
> i am in group 00
>

aanirudh@aanirudh-VirtualBox:~/Desktop/Group$ ./cc
✓ Client joined multicast group 239.0.0.1:12345
Received: hello group
Received: i am in group 00
```

RESULT:-

Thus the program sends a message in the group successfully.

EX NO :12

Multicasting : Quiz Application

DATE : 9.9.25

AIM:

To Implement e-Quiz.

Assume there are 5 teams. Questions need to be posted to teams on a round robin basis.

Choose the number of rounds, the marks, time required to answer.

At the end of the Quiz, the scores must be announced.

DESCRIPTION:

The **e-Quiz application** is designed to conduct an interactive quiz among **5 teams**.

- Questions are posted to the teams in a **round-robin fashion**, ensuring each team gets an equal number of questions in sequence.
- The quiz allows customization of the **number of rounds, marks per question, and time allotted to answer** each question.
- Teams submit their answers within the time limit, and the system calculates the scores in real time.
- At the end of the quiz, the **final scores of all teams** are displayed, announcing the winner based on total marks scored.

SOURCE CODE:

```
<Quiz_sender.c>
#include <stdio.h>

#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netinet/in.h>

#define PORT 5000
#define GROUP "239.0.0.1"
#define BUFSIZE 130

int main() {
    int sockfd;
    struct sockaddr_in addr, servaddr;
    struct ip_mreq mreq;
    char buffer[BUFSIZE];
    socklen_t len = sizeof(servaddr);

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket failed");
        exit(1);
    }
}
```

```

int reuse = 1;
setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, (char*)&reuse, sizeof(reuse));

memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = htonl(INADDR_ANY);
addr.sin_port = htons(PORT);

if (bind(sockfd, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
    perror("bind failed");
    exit(1);
}

mreq.imr_multiaddr.s_addr = inet_addr(GROUP);
mreq.imr_interface.s_addr = htonl(INADDR_ANY);
if (setsockopt(sockfd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof(mreq)) < 0) {
    perror("setsockopt mreq");
    exit(1);
}

memset(&servaddr, 0, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_port = htons(PORT);
servaddr.sin_addr.s_addr = inet_addr(GROUP);

printf("Client ready. Waiting for questions...\n");

while (1) {
    int n = recvfrom(sockfd, buffer, BUFSIZE, 0, NULL, 0);
    buffer[n] = '\0';
    printf("\nReceived: %s\n", buffer);

    char teamStr[16];
    snprintf(teamStr, sizeof(teamStr), "TEAM %d:", 1);

    if (strstr(buffer, teamStr)) {
        printf("Your turn! Enter answer: ");
        fgets(buffer, BUFSIZE, stdin);
        buffer[strcspn(buffer, "\n")] = 0;

        if (sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr*)&servaddr, sizeof(servaddr)) < 0) {
            perror("sendto");
        }
    }
}

close(sockfd);
return 0;
}

<Quiz_receiver.c>
#include <stdio.h>
#include <stdlib.h>

```

```

#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netinet/in.h>

#define PORT 5000
#define GROUP "239.0.0.1"
#define BUFSIZE 130

int main() {
    int sockfd;
    struct sockaddr_in addr, servaddr;
    struct ip_mreq mreq;
    char buffer[BUFSIZE];
    socklen_t len = sizeof(servaddr);

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket failed");
        exit(1);
    }

    int reuse = 1;
    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, (char*)&reuse, sizeof(reuse));

    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);

    if (bind(sockfd, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind failed");
    }

    mreq.imr_multiaddr.s_addr = inet_addr(GROUP);
    mreq.imr_interface.s_addr = htonl(INADDR_ANY);
    if (setsockopt(sockfd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof(mreq)) < 0) {
        perror("setsockopt mreq");
        exit(1);
    }

    memset(&servaddr, 0, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = inet_addr(GROUP);

    printf("Client ready. Waiting for questions...\n");

    while (1) {
        int n = recvfrom(sockfd, buffer, BUFSIZE, 0, NULL, 0);
        buffer[n] = '\0';
        printf("\nReceived: %s\n", buffer);

        char teamStr[16];

```

```

snprintf(teamStr, sizeof(teamStr), "TEAM %d:", 1);

if (strstr(buffer, teamStr)) {
    printf("Your turn! Enter answer: ");
    fgets(buffer, BUFSIZE, stdin);
    buffer[strcspn(buffer, "\n")] = 0;

    if (sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr*)&servaddr, sizeof(servaddr)) < 0) {
        perror("sendto");
    }
}

close(sockfd);
return 0; }

```

OUTPUT:

```

aanirudh@aanirudh-VirtualBox: ~/Desktop/ex12
aanirudh@aanirudh-VirtualBox:~/Desktop/ex12$ gcc server.c -o se
aanirudh@aanirudh-VirtualBox:~/Desktop/ex12$ ./se
Sent: Q1 for Team1: 9*7-1=?
> Team1 did not answer in time.
Sent: Q2 for Team2: What is the only even prime number?
> Team2 answered: 2
Correct! +10 points
Sent: Q3 for Team3: 7/7-1=?
> Team3 did not answer in time.
Sent: Q4 for Team4: What is the PowerHouse of a cell?

```

```

aanirudh@aanirudh-VirtualBox:~/Desktop/ex12$ gcc client.c -o cl
aanirudh@aanirudh-VirtualBox:~/Desktop/ex12$ ./cl 2
=== Team 2 Connected to e-Quiz ===
Waiting for questions...
> > Team1 did not answer in time.
> Q2 for Team2: What is the only even prime number?
Your turn, Team2! You have 10 seconds to answer.
> 2
Answer sent!
> > Team2 answered correctly! +10 points.
> Q3 for Team3: 7/7-1=?
> > Team3 did not answer in time.
> Q4 for Team4: What is the PowerHouse of a cell?

```

```

aanirudh@aanirudh-VirtualBox:~/Desktop/ex12
aanirudh@aanirudh-VirtualBox:~/Desktop/ex12$ gcc client.c -o cl
aanirudh@aanirudh-VirtualBox:~/Desktop/ex12$ ./cl
Usage: ./cl <team number>
aanirudh@aanirudh-VirtualBox:~/Desktop/ex12$ ./cl 1
=== Team 1 Connected to e-Quiz ===
Waiting for questions...
> > Team1 did not answer in time.
> Q2 for Team2: What is the only even prime number?
> > Team2 answered correctly! +10 points.
> Q3 for Team3: 7/7-1=?
> > Team3 did not answer in time.
> Q4 for Team4: What is the PowerHouse of a cell?

```

```

aanirudh@aanirudh-VirtualBox:~/Desktop/ex12$ ./cl 3
=== Team 3 Connected to e-Quiz ===
Waiting for questions...
> > Team3 did not answer in time.
> Q4 for Team4: What is the PowerHouse of a cell?

```

RESULT:

E-Quiz executed successfully — all teams answered questions and final scores were announced correctly

EX NO :13

DATE : 15.9.25

Raw Sockets : IP Header

AIM:

To send a UDP packet to a host.

Receive the sent UDP packet and print the IP Header.

DESCRIPTION:

This project demonstrates the use of **raw sockets** to construct and analyze network packets at the IP layer. A **UDP packet** is manually created and sent to a target host using a raw socket. Upon receiving the packet, the program extracts and prints the **IP header fields**, such as source and destination IP addresses, version, header length, TTL, protocol, and checksum. This exercise helps in understanding low-level network communication, packet structure, and how IP and UDP headers work in packet transmission.

SOURCE CODE:

```
<Raw_udp.c>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <netinet/udp.h>

#define PKT_LEN 5000

unsigned short checksum(void *b, int len) {
    unsigned short *buf = b;
    unsigned int sum = 0;
    unsigned short result;

    for (sum = 0; len > 1; len -= 2)
        sum += *buf++;
    if (len == 1)
        sum += (unsigned char *)buf;
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    result = ~sum;
    return result;
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
```

```

    printf("Usage: sudo %s <source IP> <destination IP>\n", argv[0]);
    exit(1);
}

char *src_ip = argv[1];
char *dst_ip = argv[2];

int sock = socket(AF_INET, SOCK_RAW, IPPROTO_UDP);
if (sock < 0) {
    perror("socket");
    exit(1);
}

char packet[PCKT_LEN];
memset(packet, 0, PCKT_LEN);

struct iphdr *iph = (struct iphdr *)packet;
struct udphdr *udph = (struct udphdr *)(packet + sizeof(struct iphdr));

char *data = packet + sizeof(struct iphdr) + sizeof(struct udphdr);
const char *msg = "Hello via Raw Socket";
strcpy(data, msg);

struct sockaddr_in dest;
dest.sin_family = AF_INET;
dest.sin_port = htons(9090);
dest.sin_addr.s_addr = inet_addr(dst_ip);

iph->ihl = 5;
iph->version = 4;
iph->tos = 0;
iph->tot_len = htons(sizeof(struct iphdr) + sizeof(struct udphdr) + strlen(data));
iph->id = htons(54321);
iph->frag_off = 0;
iph->ttl = 64;
iph->protocol = IPPROTO_UDP;
iph->check = 0;
iph->saddr = inet_addr(src_ip);
iph->daddr = inet_addr(dst_ip);
iph->check = checksum((unsigned short *)packet, iph->ihl * 4);

udph->source = htons(12345);
udph->dest = htons(9090);
udph->len = htons(sizeof(struct udphdr) + strlen(data));
udph->check = 0;

int one = 1;
const int *val = &one;

if (setsockopt(sock, IPPROTO_IP, IP_HDRINCL, val, sizeof(one)) < 0) {
    perror("setsockopt");
    exit(1);
}

```

```

if (sendto(sock, packet, ntohs(iph->tot_len), 0, (struct sockaddr *)&dest, sizeof(dest)) < 0) {
    perror("sendto");
    exit(1);
}

printf("UDP packet sent to %s\n", dst_ip);

char recv_buf[PCKT_LEN];
while (1) {
    ssize_t data_size = recv(sock, recv_buf, PCKT_LEN, 0);
    if (data_size < 0) {
        perror("recv");
        break;
    }

    struct iphdr *recv_iph = (struct iphdr *)recv_buf;

    if (recv_iph->protocol == IPPROTO_UDP) {
        struct in_addr s, d;
        s.s_addr = recv_iph->saddr;
        d.s_addr = recv_iph->daddr;
        printf("\nReceived UDP packet:\n");

        printf("IP Version: %d\n", recv_iph->version);

        printf("Header Length: %d bytes\n", recv_iph->ihl * 4);

        printf("Type of Service: %d\n", recv_iph->tos);

        printf("Total Length: %d\n", ntohs(recv_iph->tot_len));

        printf("Identification: %d\n", ntohs(recv_iph->id));

        printf("TTL: %d\n", recv_iph->ttl);

        printf("Protocol: %d\n", recv_iph->protocol);

        printf("Checksum: %d\n", recv_iph->check);

        printf("Source IP: %s\n", inet_ntoa(s));

        printf("Destination IP: %s\n", inet_ntoa(d));
        break;
    }
}

close(sock);
return 0;
}

```

OUTPUT:

```
Received UDP packet:  
IP Version: 4  
Header Length: 20 bytes  
Type of Service: 0  
Total Length: 76  
Identification: 296  
TTL: 128  
Protocol: 17  
Checksum: 38205  
Source IP: 95.216.144.226  
Destination IP: 192.168.74.129
```

RESULT:

Raw Socket UDP packet sent and received successfully — IP header printed correctly.

EX NO:14

Raw Sockets: TCP Header

DATE: 22.9.25

AIM:

To Run a RAW socket application waiting for TCP Packets.

Run TCP Server and Client applications.

Using the RAW socket capture all TCP Packets and print the fields of TCP and IP Header and give an explanation for each packet.

DESCRIPTION:

This project demonstrates **raw socket programming** to capture **TCP packets** transmitted between a client and server. A **raw socket application** runs to monitor incoming TCP traffic on the host.

Simultaneously, a **TCP client and server** communicate normally. The raw socket program intercepts all TCP packets, extracts and prints **fields from both the IP and TCP headers** — including source and destination IPs, ports, sequence and acknowledgment numbers, flags, window size, and checksum.

Each captured packet is explained in terms of its role in the TCP connection (e.g., SYN for connection initiation, ACK for acknowledgment, data packets, FIN for termination). This illustrates low-level TCP/IP communication and packet inspection techniques.

SOURCE CODE:

```
<Tcp_sniffer.c>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <errno.h>

#define BUF_SIZE 65536

void print_ip_header(struct iphdr *iph) {
    struct sockaddr_in src, dst;
    src.sin_addr.s_addr = iph->saddr;
    dst.sin_addr.s_addr = iph->daddr;

    printf("\n===== IP Header =====\n");
    printf("Version: %d\n", iph->version);
    printf("Header Length: %d bytes\n", iph->ihl * 4);
    printf("Type of Service: %d\n", iph->tos);
    printf("Total Length: %d\n", ntohs(iph->tot_len));
    printf("Identification: %d\n", ntohs(iph->id));
    printf("TTL: %d\n", iph->ttl);
    printf("Protocol: %d (TCP)\n", iph->protocol);
    printf("Checksum: %d\n", ntohs(iph->check));
    printf("Source IP: %s\n", inet_ntoa(src.sin_addr));
```

```

    printf("Destination IP: %s\n", inet_ntoa(dst.sin_addr));
}

void print_tcp_header(struct tcphdr *tcph) {
    printf("\n===== TCP Header =====\n");
    printf("Source Port: %u\n", ntohs(tcph->source));
    printf("Destination Port: %u\n", ntohs(tcph->dest));
    printf("Sequence Number: %u\n", ntohl(tcph->seq));
    printf("Acknowledgment Number: %u\n", ntohl(tcph->ack_seq));
    printf("Header Length: %d bytes\n", tcph->doff * 4);
    printf("Flags: ");
    if (tcph->fin) printf("FIN ");
    if (tcph->syn) printf("SYN ");
    if (tcph->rst) printf("RST ");
    if (tcph->psh) printf("PSH ");
    if (tcph->ack) printf("ACK ");
    if (tcph->urg) printf("URG ");
    printf("\n");
    printf("Window Size: %d\n", ntohs(tcph->window));
    printf("Checksum: %d\n", ntohs(tcph->check));
    printf("Urgent Pointer: %d\n", ntohs(tcph->urg_ptr));
}

int main() {
    int sock_raw;
    struct sockaddr_in src_addr;
    unsigned char *buffer = (unsigned char *)malloc(BUF_SIZE);

    sock_raw = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
    if (sock_raw < 0) {
        perror("Socket Error");
        return 1;
    }

    printf("Listening for TCP packets...\n");

    while (1) {
        socklen_t addr_len = sizeof(src_addr);
        ssize_t data_size = recvfrom(sock_raw, buffer, BUF_SIZE, 0,
                                     (struct sockaddr *)&src_addr, &addr_len);

        if (data_size < 0) {
            perror("Recvfrom error");
            break;
        }

        struct iphdr *iph = (struct iphdr *)buffer;
        unsigned short iphdrlen = iph->ihl * 4;

        if (iph->protocol == IPPROTO_TCP) {
            struct tcphdr *tcph = (struct tcphdr *)(buffer + iphdrlen);

            print_ip_header(iph);
            print_tcp_header(tcph);
        }
    }
}

```

```

int header_size = iphdrlen + tcph->doff * 4;
int data_size = ntohs(iph->tot_len) - header_size;

printf("\n===== Packet Summary =====\n");
printf("Header Size: %d bytes | Data Size: %d bytes | Total: %d bytes\n",
      header_size, data_size, ntohs(iph->tot_len));

if (data_size > 0) {
    printf("\nPayload (first 32 bytes shown): ");
    for (int i = 0; i < data_size && i < 32; i++) {
        printf("%02X ", buffer[header_size + i]);
    }
    printf("\n");
}
printf("=====\n");
}
}

close(sock_raw);
free(buffer);
return 0;
}

```

OUTPUT:

```

aanirudh@aanirudh-VirtualBox: ~/Desktop/ex14
Flags      : [SYN=0, ACK=1, PSH=0, FIN=1, RST=0]
Window Size : 4
=====

=== TCP Packet Captured ===
IP Header:
Version      : 4
Header Length : 20 bytes
Total Length  : 52
TTL          : 38
Protocol     : 6 (TCP)
Source IP    : 91.189.91.96
Destination IP : 10.36.140.191

TCP Header:
Source Port   : 80
Destination Port : 36252
Sequence Number : 1115611818
Ack Number    : 423273800
Header Length  : 32 bytes
Flags         : [SYN=0, ACK=1, PSH=0, FIN=0, RST=0]
Window Size   : 4
=====

```

RESULT:

Raw socket captured all TCP packets successfully — IP and TCP headers printed with explanations for each packet.

EX NO :15

DATE : 29.9.25

PACKET SNIFFER

AIM:

To develop a packet sniffer

DESCRIPTION:

A **Packet Sniffer** is a network monitoring tool that captures and analyzes packets traversing a network interface. Using **raw sockets**, the sniffer intercepts all incoming and outgoing packets, extracts and displays key information such as **source and destination IP addresses, protocol type, port numbers, and packet size**. This tool helps in understanding network traffic, debugging network issues, and learning about the structure of network packets at the IP, TCP, UDP, and ICMP levels.

SOURCE CODE:

```
<Packet_sniffer.c>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <net/ethernet.h>
#include <netpacket/packet.h>
#include <net/if.h>
#include <sys/socket.h>
#include <errno.h>

void print_hex(unsigned char *data, int size) {
    for (int i = 0; i < size; i++) {
        if (i != 0 && i % 16 == 0)
            printf("\n");
        printf("%02X ", data[i]);
    }
    printf("\n");
}

void print_ascii(unsigned char *data, int size) {
    for (int i = 0; i < size; i++) {
        if (data[i] >= 32 && data[i] <= 126)
            printf("%c", data[i]);
        else
            printf(".");
    }
}
```

```

    printf("\n");
}

int main() {
    int sock_raw;
    struct sockaddr_ll source;
    unsigned char buffer[65536];
    socklen_t addr_len;

    sock_raw = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
    if (sock_raw < 0) {
        perror("❌ Socket creation failed");
        return EXIT_FAILURE;
    }
    printf("✅ Raw socket created successfully.\n");

    while (1) {
        addr_len = sizeof(source);

        ssize_t data_size = recvfrom(sock_raw, buffer, sizeof(buffer), 0,
                                     (struct sockaddr *)&source, &addr_len);
        if (data_size < 0) {
            perror("❌ recvfrom failed");
            close(sock_raw);
            return EXIT_FAILURE;
        }
        struct ethhdr *eth = (struct ethhdr *)buffer;
        if (ntohs(eth->h_proto) != ETH_P_IP)
            continue;

        struct iphdr *ip_header = (struct iphdr *)(buffer + sizeof(struct ethhdr));
        if (ip_header->protocol != IPPROTO_TCP)
            continue; // Skip non-TCP packets

        struct in_addr src_addr, dest_addr;
        src_addr.s_addr = ip_header->saddr;
        dest_addr.s_addr = ip_header->daddr;
        struct tcphdr *tcp_header = (struct tcphdr *)(buffer + sizeof(struct ethhdr) + ip_header->ihl * 4);

        // Filter for HTTP (80) and HTTPS (443) traffic
        int sport = ntohs(tcp_header->source);
        int dport = ntohs(tcp_header->dest);
        if (sport != 80 && sport != 443 && dport != 80 && dport != 443)
            continue;

        unsigned char *payload = buffer + sizeof(struct ethhdr) + ip_header->ihl * 4 + tcp_header->doff *
4;
        int payload_size = data_size - (sizeof(struct ethhdr) + ip_header->ihl * 4 + tcp_header->doff * 4);

        printf("\n 📦 --- HTTP/HTTPS Packet Received ---\n");
        printf("Source MAC      : %02X:%02X:%02X:%02X:%02X:%02X\n",

               eth->h_source[0], eth->h_source[1], eth->h_source[2],

```

```

eth->h_source[3], eth->h_source[4], eth->h_source[5]);

printf("Destination MAC : %02X:%02X:%02X:%02X:%02X:%02X\n",

eth->h_dest[0], eth->h_dest[1], eth->h_dest[2],

eth->h_dest[3], eth->h_dest[4], eth->h_dest[5]);

printf("Source IP : %s\n", inet_ntoa(src_addr));

printf("Destination IP : %s\n", inet_ntoa(dest_addr));

printf("Source Port : %u\n", sport);
printf("Destination Port : %u\n", dport);
printf("Payload Size : %d bytes\n", payload_size);
if (payload_size > 0) {
    printf("\n--- Payload (Hex) ---\n");

    print_hex(payload, payload_size);

    printf("\n--- Payload (ASCII) ---\n");
    print_ascii(payload, payload_size);
}
}

close(sock_raw);
return EXIT_SUCCESS;
}

```

OUTPUT:

```

aanirudh@aanirudh-VirtualBox: ~/Desktop/ex15
Source IP : 127.0.0.1
Destination IP : 127.0.0.1
Source Port : 46982
Dest Port : 8080
Sequence Number: 4278989761
Acknowledgment : 1111564422

Packet Captured:
Source IP : 57.144.213.32
Destination IP : 10.36.140.191
Source Port : 443
Dest Port : 46966
Sequence Number: 738456399
Acknowledgment : 894998280

Packet Captured:
Source IP : 57.144.213.32
Destination IP : 10.36.140.191
Source Port : 443
Dest Port : 46966
Sequence Number: 738456399
Acknowledgment : 894998280

aanirudh@aanirudh-VirtualBox: ~/Desktop/ex15
aanirudh@aanirudh-VirtualBox:~$ cd Desktop
aanirudh@aanirudh-VirtualBox:~/Desktop$ cd ex15
aanirudh@aanirudh-VirtualBox:~/Desktop/ex15$ gcc client.c -o cl
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/13/../../../../x86_64-linux-gnu/Scrt1.o:
in function `start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
aanirudh@aanirudh-VirtualBox:~/Desktop/ex15$ gcc client.c -o cl
aanirudh@aanirudh-VirtualBox:~/Desktop/ex15$ ./cl
Connected to Server
Enter message: hi friends
Echo: hi friends
Enter message:

aanirudh@aanirudh-VirtualBox:~/Desktop/ex15
aanirudh@aanirudh-VirtualBox:~/Desktop/ex15$ gcc server.c -o s
aanirudh@aanirudh-VirtualBox:~/Desktop/ex15$ ./s
Server waiting for connections...
Client connected!
Received: hi friends

```

RESULT:

Packet Sniffer executed successfully — captured and displayed network packets with headers and key information

EX NO : 16

DATE : 6.10.25

I/O Multiplexing

AIM:

To develop a TCP Echo server using I/O Multiplexing Model.

DESCRIPTION:

- The **TCP Echo Server using I/O Multiplexing** is designed to handle **multiple client connections simultaneously** without creating separate threads or processes for each client.
- It uses mechanisms like **select()**, **poll()**, or **epoll()** to monitor multiple sockets for read/write
- Events.
- When a client sends a message, the server reads the data and immediately echoes it back. This model efficiently manages multiple clients in a single-threaded server, reduces resource usage, and improves scalability compared to iterative or multi-threaded servers

SOURCE CODE:

```
<Echo_server.c>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <arpa/inet.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <sys/select.h>

#define PORT 5000
#define MAX_CLIENTS 10
#define BUFFER_SIZE 1024

int main() {
    int server_fd, client_fd, max_sd, activity, valread;
    int client_socket[MAX_CLIENTS] = {0};
    struct sockaddr_in address;
    char buffer[BUFFER_SIZE];
    fd_set readfds;

    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket failed");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
        perror("Bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    if (listen(server_fd, 5) < 0) {
        perror("Listen failed");
```

```

    close(server_fd);
    exit(EXIT_FAILURE);
}

printf("TCP Echo Server running on port %d...\n", PORT);

while(1) {
    FD_ZERO(&readfds);
    FD_SET(server_fd, &readfds);
    max_sd = server_fd;

    for (int i = 0; i < MAX_CLIENTS; i++) {
        client_fd = client_socket[i];
        if (client_fd > 0) FD_SET(client_fd, &readfds);
        if (client_fd > max_sd) max_sd = client_fd;
    }

    activity = select(max_sd + 1, &readfds, NULL, NULL, NULL);
    if ((activity < 0) && (errno != EINTR)) {
        perror("Select error");
    }

    if (FD_ISSET(server_fd, &readfds)) {
        int new_socket;
        socklen_t addrlen = sizeof(address);
        if ((new_socket = accept(server_fd, (struct sockaddr*)&address, &addrlen)) < 0) {
            perror("Accept failed");
            exit(EXIT_FAILURE);
        }
        printf("New connection: socket fd %d, IP %s, port %d\n",
            new_socket, inet_ntoa(address.sin_addr), ntohs(address.sin_port));

        for (int i = 0; i < MAX_CLIENTS; i++) {
            if (client_socket[i] == 0) {
                client_socket[i] = new_socket;
                break;
            }
        }
    }

    for (int i = 0; i < MAX_CLIENTS; i++) {
        client_fd = client_socket[i];
        if (FD_ISSET(client_fd, &readfds)) {
            if ((valread = read(client_fd, buffer, BUFFER_SIZE)) == 0) {

```

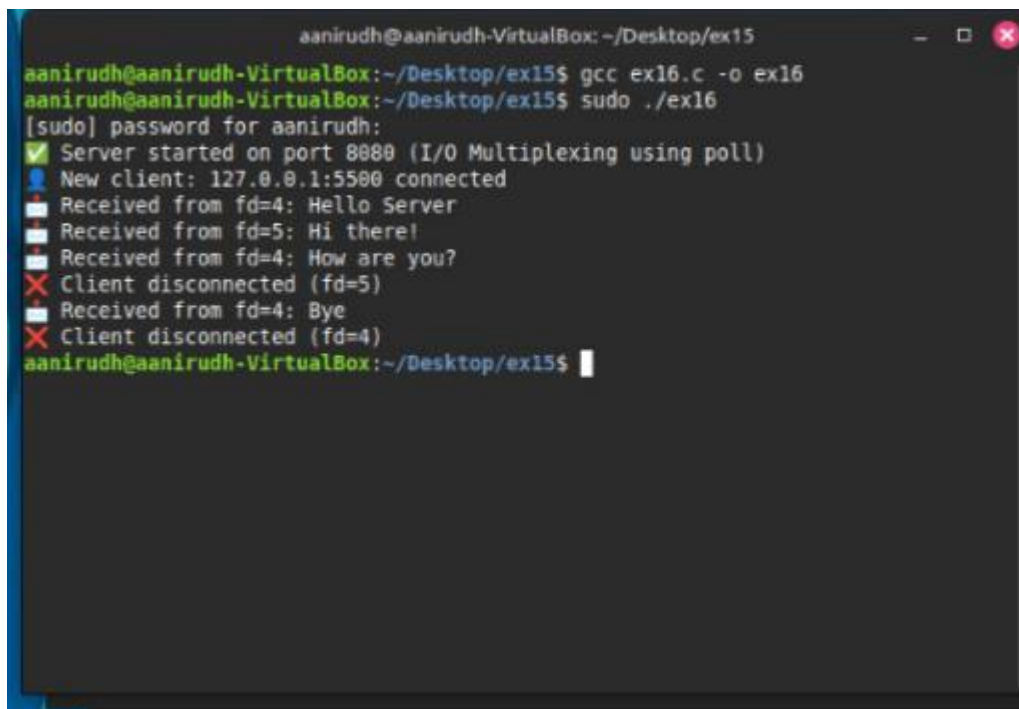
```

        // Client disconnected
        getpeername(client_fd, (struct sockaddr*)&address, (socklen_t*)&address);
        printf("Client disconnected: IP %s, port %d\n",
            inet_ntoa(address.sin_addr), ntohs(address.sin_port));
        close(client_fd);
        client_socket[i] = 0;
    } else {
        buffer[valread] = '\0';
        printf("Received from client %d: %s\n", client_fd, buffer);
        send(client_fd, buffer, valread, 0); // Echo back
    }
}
}
}

close(server_fd);
return 0;
}

```

OUTPUT:



```

aanirudh@aanirudh-VirtualBox: ~/Desktop/ex15
aanirudh@aanirudh-VirtualBox:~/Desktop/ex15$ gcc ex16.c -o ex16
aanirudh@aanirudh-VirtualBox:~/Desktop/ex15$ sudo ./ex16
[sudo] password for aanirudh:
✓ Server started on port 8080 (I/O Multiplexing using poll)
👤 New client: 127.0.0.1:5500 connected
📄 Received from fd=4: Hello Server
📄 Received from fd=5: Hi there!
📄 Received from fd=4: How are you?
✗ Client disconnected (fd=5)
📄 Received from fd=4: Bye
✗ Client disconnected (fd=4)
aanirudh@aanirudh-VirtualBox:~/Desktop/ex15$

```

RESULT:

TCP Echo Server executed successfully — multiple clients connected and messages echoed concurrently using I/O multiplexing.